Revision Summary and Detailed Response for Paper 201

MemForest: An Efficient Agent Memory System with Hierarchical Temporal Indexing

Dear Reviewers, Meta-Reviewer, and Associate Editor,

Thank you for the detailed comments. We revised the paper while preserving its architecture and central objective: reducing the freshness cost of persistent agent memory without sacrificing answer quality. Blue text marks substantive changes. All reviewer comments below are quoted verbatim. Due to the paper-length limit, detailed diagnostics, configurations, and supplementary tables are provided in Appendices A–F of the public complete version at reproducibility/paper/MemForest_full_revision.pdf. Code and result artifacts are hosted at https://github.com/Concyclics/MemForest; every repository reference below prints its clickable path explicitly.

REVISION SUMMARY AND RESPONSE TO THE META-REVIEW

This section also serves as the revision summary. Detailed evidence is not repeated here; each item refers to the corresponding reviewer response.

C-M1. *Strengthen the positioning and motivation of MemTree. Justify the need for a hierarchical temporal memory structure and compare it against simpler temporal alternatives (e.g., timestamped logs, validity intervals, temporal graphs or semantic timelines).*

Response. Section 7 and Table 1 position simpler temporal designs; Section 6.2 and Figure 8(e) compare them under shared facts and budgets; Tables 2–3 add native Zep Local. For details, see R3-W1 for the controlled comparison, R3-W4 for Zep Local, and R4-D2–D3 for scope and recent/background controls.

C-M2. *Clarify the write-path cost model and complexity analysis. Provide a detailed breakdown of the memory construction and maintenance pipeline, justify the assumptions underlying the complexity analysis (e.g., balanced-tree), and identify the dominant cost components.*

Response. Sections 4.1, 4.3, 4.4, and 5.4 separate extraction, insertion, and refresh and limit the height bound to structural/dirty-path depth. Table 1 compares complete-build LLM-wave depth across released implementations. Figures 7 and 8(c–d) report balance/scaling and stage costs; Section 6.1 and Figure 8(a–b) report the latency–token–fidelity control. For details, see R1-W1 and R3-W2 for trade-off/cost, and R4-D1, R4-D6–D7, and R4-D16 for balance, chunking, prefill, and scaling.

C-M3. *Provide a stronger and fairer empirical evaluation. Include additional temporal-memory baselines (such as Zep and simpler temporal alternatives), clarify timestamp handling and baseline configurations and reconcile discrepancies with previously reported results. The evaluation should characterize the quality/efficiency trade-off by jointly considering answer quality, write latency, query latency, token consumption, and LLM-call costs. Scalability should also be characterized (as memory size and the number of MemTrees grow).*

Response. Sections 5.1– 5.5, Tables 2 and 3, and Figures 1, 5, 7, and 8(c–d) report the aligned protocol, Zep/Gemma additions, quality, write/query cost, and scaling. The Appendix D.1 (token diagnostics) and Appendix D.2 (query-scaling diagnostics) provide full traces. For details, see R1-W1–W2, R3-W1–W2/R3-W5–W6, and R4-D14–D16 for trade-offs, protocol reconciliation, and scaling.

C-M4. *Validate routing, retrieval, and temporal-memory robustness. Quantify the impact of routing decisions, planner behavior, evidence fragmentation, incorrect scope assignments and retrieval errors on final answer quality. The revision should also clarify how scope conflicts are handled.*

Response. Sections 4.1, 4.2, and 6.2 define multi-scope routing/union recall and report routing, fragmentation, and scene stability; Tables 6–7 isolate scope and planner effects. Full labels are in the Appendix B.4 (routing diagnostics). For details, see R3-W3 and R4-D2/R4-D4–D5/R4-D12.

C-M5. *Clarify system semantics and implementation details. Provide detail regarding canonicalization, conflict resolution, retrieval scoring, tree traversal, merge policies and the handling of extraction errors. The revision should also specify freshness and consistency semantics.*

Response. Sections 4.1–4.3 define canonicalization/conflicts, scoring/indexing, traversal, and refresh; Algorithm 1 and Section 4.5 define maintenance, visibility, locks, and snapshots. The Appendix D.3 (freshness and write-concentration analysis) reports cross-session lock overlap. Sections 5.6 and 6.1 cover merge and extraction fidelity. For details, see R3-W3 and R4-D4/R4-D8–D13 for the corresponding semantics and code paths.

C-M6. *Discuss the generality of the results beyond Qwen models.*

Response. Section 5.1 adds a full Gemma-4-12B-IT generative setting; Tables 2 and 3 report all methods under three backbones. R4-D14 discusses these tables and the cross-family conclusion.

C-M7. *Ensure full reproducibility. Make the experimental artifacts available and ensure that the evaluation complies with the conference reproducibility requirements (you can also find a precise list from one of the reviewers).*

Response. The first page gives the public artifact URL. We release the reference implementation and rerun scripts for the evaluated components, together with core execution logs, per-question outputs, manifests, validators, and the data underlying the main results. Reproducing model-serving runs requires compatible external endpoints and the licensed benchmark inputs. The artifact index is reproducibility/README.md. For details, see R1-W3 and R3-W5–W6 for release/evaluation artifacts, and R4-D8/R4-D10–D13 for line-level implementation links.

DETAILED RESPONSE TO REVIEWER #1

R1-W1. *It would be better if Section 5.2 explains the latency-token tradeoff more, such as how to control the tradeoff from a practical perspective.*

Response. Yes. We now answer the existence, cause, and control of this trade-off in Sections 4.1 and 6.1, supported by Figure 8(a–b).

Existence and cause. Revised Section 4.1 states:

> *This creates a latency–token trade-off: smaller chunks expose more parallel requests and bound each generation, but repeat prompt/schema input and JSON scaffolding; larger chunks amortize that overhead but reduce parallelism and can lose answer-bearing evidence as context grows [18]. For short sessions or below serving saturation, repeated prefill can outweigh the parallel-latency benefit.*

Control. Revised Section 6.1 states:

> *Figure 8(a–b) sweeps chunk size on an assembled long session. Larger chunks reduce total tokens chiefly by extracting fewer facts: Gold Coverage falls and tokens/fact eventually worsens, consistent with lost-in-the-middle effects [18]. We select $b = 2$ because it preserves 100% Gold Coverage, remains near peak throughput, and uses fewer tokens per fact than $b = 1$. In deployment, the largest chunk that preserves fidelity and exposes enough parallel requests provides the practical control.*

R1-W2. *I wonder why LoCoMo is the harder benchmark for MemForest than LongMemEval. Section 2.3.3 says that LoCoMo has more temporal-reasoning questions than LongMemEval-S, and because MemForest focuses on temporal management, I expected that MemForest would have the best pass@1 accuracy also in LoCoMo. The authors mention that LoCoMo has more entangled evidence than LongMemEval, but more explanations would be helpful.*

Response. The submitted discussion was based on an incorrect category mapping: under the official mapping, temporal questions are category 2 (321/1,986, or 16.2%), whereas the 841-question block is single-hop. We removed the resulting benchmark-difficulty comparison from revised Section 2.2.2 and apologize for the labeling error.

We also align the headline scope with the released Mem0/EverMemOS protocol by reporting categories 1–4 (1,540 questions) in Table 3 and evaluating adversarial category 5 separately in the Appendix A.4 (LoCoMo category-5 results). Judge-policy sensitivity is addressed separately in R3-W6.

R1-W3. *It would be better if source codes and experimental setups were available.*

Response. We apologize for this omission. The repository URL was present in the submission system but was omitted from the submitted manuscript. The revised first page now states:

> *PVLDB Artifact Availability: The source code, data, and/or other artifacts have been made available at https://github.com/Concyclics/MemForest.*

Revised Section 5.1 further states:

> *Configurations and validation records are available in the baseline guide and Appendix A.2.*

The repository paths are reproducibility/BASELINES.md and reproducibility/manifests/baseline_versions.json.

DETAILED RESPONSE TO REVIEWER #3

R3-W1. *Target scenarios need sharper justification. The temporal motivation is strong, but MemTree itself is not fully justified. Simpler designs such as timestamped logs, validity intervals, temporal graphs, or semantic timelines might also handle the example queries. The paper should compare with stronger temporal baselines, at least a timestamp-aware log and a temporal graph/timeline method.*

Response. Revised Sections 7 and 6.2 separate native end-to-end systems from shared-fact representation controls, summarized below.

Reviewer design	Native end-to-end baseline	Shared-fact control
Timestamped log / timeline	MemPalace; EverMemOS	Flat Log; Log-Time
Recent scratchpad + background index	LightMem; MemoryOS	Scratchpad+Background
Validity intervals	Zep/Graphiti bi-temporal validity	Validity Interval
Temporal graph	Zep Local / Graphiti	Triplet Graph

Tables 2 and 3 report the native systems, including Zep Local. Figure 8(e) then reports the controlled comparison described in the paper:

> *All methods share facts, timestamps, embeddings, and budgets; selectors tune only on development data. Log-Time leads MemTree by 0.7 points at $k = 10$; they are within 0.1 points at $k = 30$, and MemTree leads at $k = 50$ (92.0%). This places MemTree as a high-recall option at larger budgets, while flat logs avoid summary construction.*

R3-W2. *Efficiency needs a clearer breakdown. The paper risks making updates sound logarithmic, but the write path still includes LLM extraction, routing, embedding, refresh, and index maintenance. These costs should be separated more clearly, including when LLM work or dirty-node refresh dominates. The LoCoMo results also need a clearer quality–efficiency comparison, especially against EverMemOS.*

Response. Revised Section 5.4 and Table 1 first separate logical dependency depth from total work. The revised paper states:

> *Bounded extraction yield and balanced bounded fan-out give $D = O(N)$ and $H = O(\log N)$, hence $O(\lceil N/P \rceil + \log N)$. This is dependency depth, not total work: fixed P gives $O(N)$, and token/non-LLM costs remain. The evaluated Mem0, MemoryOS, EverMemOS, LightMem, and Zep Local paths retain an $O(N)$ LLM chain; MemPalace has zero LLM depth but $O(N)$ indexing.*

The same section and Figure 8(c–d) then report:

> *Autoregressive extraction and dirty refresh account for 83.58% of measured build time; including hybrid canonicalization raises the LLM-involved share to 90.57%, while structural insertion, index update, and*

> *persistence together remain below 1%. Parent summaries depend on refreshed children, whereas same-level and cross-tree calls form independent waves. A throughput-calibrated serial replay estimates 54.58 and 9.88 minutes for extraction and dirty refresh, compared with measured parallel times of 122.31 and 27.92 seconds (26.8× and 21.2× speedups).*

The Appendix E (detailed write-path replay and dependency analysis) reports the replay model, sensitivity range, and method-by-method dependencies; the Appendix D.1 (request/token diagnostics) reports calls and per-request tokens. Released trace inputs are at reproducibility/results/write_path_traces/.

End-to-end construction is not $O(\log N)$. Revised Section 4.4 states the narrower bound directly:

> *Under bounded fan-out and balanced MemTrees, structural insertion and dirty-path dependency depth are $O(\log_k N)$; extraction scales with incoming content, and refresh work scales with D.*

Figure 1 provides the requested LongMemEval and LoCoMo quality–efficiency comparison, including EverMemOS and Zep Local.

R3-W3. *Retrieval robustness and freshness are unclear. MemForest relies on routing facts into the right trees, but the paper does not measure routing or scene-clustering errors. The lazy-refresh design also needs clearer semantics: what becomes visible after a write, whether summaries may be stale, and how concurrent reads and writes are handled.*

Response. We address this comment in three aspects.

(1) Routing accuracy and scene stability. Revised Section 6.2 reports the reviewed routing, fragmentation, and scene-threshold results; the Appendix B.4 (routing and fragmentation diagnostics) provides the underlying measurements and labels. The revised paper concludes:

> *A manual audit finds 124/127 (97.6%) entity assignments valid but only 9/127 complete. Fragmentation lowers embedding-browse accuracy from 76.2% to 50.0% on LoCoMo and 78.1% to 67.6% on LongMemEval. Scene-threshold Jaccard mean/p05 is at least 0.9997/0.9996 (4B) and 0.9650/0.8829 (30B). These results support multi-scope placement and browse.*

Reviewed routing labels and fragmentation summaries are available at reproducibility/results/semantic_audit/author_adjudicated/.

(2) Lazy refresh. Revised Section 4.3 states:

> *Within each write batch, records are grouped by touched tree and affected ancestors are marked dirty. Dirty nodes are deduplicated, grouped by level, and refreshed bottom-up before the write returns.*

Thus, "lazy" refers to batched summary refresh, not to stale visibility after the completed write.

The Appendix D.3 (freshness and write-concentration analysis) reports cross-session overlap with and without the global `entity:user` fallback; Section 5.6 provides the subforest migration/merge path used to shard that fallback. The released audit path is reproducibility/results/write_conflicts/.

(3) Concurrency and visibility. Revised Section 4.5 states:

> *Each MemTree owns an independent reader–writer lock. Insertion and summary refresh take the touched tree's write lock, so distinct trees proceed concurrently while same-tree writes serialize. Range queries, browse, and snapshot export use shared read locks; fact-store and root-index updates use separate component locks. The online* `auto_update` *path refreshes dirty summaries and root vectors before returning, and snapshot save atomically renames a completed temporary directory into place.*

The implementation directory is reproducibility/implementation/async_core/. The relevant clickable locations are `bplustree.py`, lines 21–54 for the reader–writer lock, `forest.py`, lines 1425–1476 for cross-tree insertion and dirty-set snapshotting, and `forest.py`, lines 2036–2106 for atomic snapshot publication.

R3-W4. *W4: §2.3 overstates the limitations of prior work. §2.3 claims existing systems cannot answer historical-state ("what was true before X") queries. This holds for an idealized pure-vector or pure-profile design, but not for the temporally-structured systems the paper also cites and benchmarks. Zep [1] solves it by construction—a bi-temporal graph where superseded edges are invalidated but kept, with a non-lossy episodic store. Yet it is cited and excluded from the experiments: the most temporally capable baseline is omitted while the paper claims existing systems can't do temporal. The authors' own traces [2] refute the premise. The released EverMemOS answers retrieve the correct, correctly-time-stamped evidence on essentially every temporal question (e.g., "[April 6, 2023] attended the Maundy Thursday service"). The historical state is present and retrieved—not lost.*

Response. Please refer to R3-W1 for the full representation mapping and controlled comparison. We corrected Section 2.2.2 to describe the mechanisms used by the evaluated temporal baselines rather than claim that prior systems cannot preserve historical evidence:

> *Existing baselines preserve temporal evidence through concrete, different mechanisms. EverMemOS keeps relative time expressions together with resolved absolute dates in episodic and event memories, then uses LLM-guided multi-round retrieval [13].*

The same paragraph maps MemPalace, LightMem/MemoryOS, and Zep/Graphiti to their timestamped-history, tiered-memory, and bi-temporal-graph mechanisms. We also add both an end-to-end Zep Local baseline and a controlled representation comparison over timestamped logs, validity intervals, recent-memory/background consolidation, a triplet graph, and MemTree. Revised Section 5.1 states:

> *Following Zhou et al. [44], Zep Local is implemented with the open Graphiti temporal-graph core, Neo4j, and self-hosted generation and embedding services. It represents the local Graphiti design, not the Zep Cloud service.*

Tables 2 and 3 report the added end-to-end results. The Appendix A.2 (Zep Local protocol and native-object budget) specifies the pinned Graphiti/Neo4j recipe and native retrieval objects; Section 6.2 reports the shared-fact controlled comparison. The released Zep Local runner is reproducibility/baselines/zep_local/run_benchmark.py. We also revise related work to state explicitly that MemPalace and EverMemOS retain timestamped histories.

R3-W5. *W5: The temporal-category gap might be a timestamp-configuration artifact, not the substrate. Mem0: the conversation timestamps seem not to be stored with the event. The dates in Mem0's Temporal benchmark outputs seem to be 2026, instead of the event's actual date 2023. If the event is not carried with a timestamp, every "how many days ago" question is unanswerable. This is configuration, not capability: Mem0's* `timestamp` *parameter has existed at-least since late-2025. [3] Scale: A lot of LongMemEval temporal questions are date arithmetic ("how many days/weeks ago"), so these issues drive the category score. [2] Conclusion: the comparison conflates substrate quality with per-system timestamp/reference-date handling, so the temporal win cannot be attributed to MemTree.*

Response. We thank the reviewer for identifying this issue. We audited timestamp propagation in all evaluated adapters and found this specific benchmark-to-runtime mismatch only in our submitted Mem0 adapter. For the evaluated local-REST path, benchmark session time is normalized to UTC and written to `metadata.created_at` and `metadata.event_time`; add batches are bounded within one source session. The SDK fallback passes the same time through Mem0's `timestamp` argument. All reported reruns use the local-REST path, complete message-coverage checks, runtime-date rejection, and a global top-10 flat retrieval budget. The refreshed results replace the submitted Mem0 values in Tables 2 and 3. The released adapter preserving this correction is at reproducibility/baselines/mem0/mem0_adapter.py; the rerun manifest records both evaluated and released adapter hashes, and its validation summary is at reproducibility/results/mem0_corrected/.

R3-W6. *W6: Baseline temporal scores fall far below the systems' own published numbers, and only on temporal. The baselines here underperform their own reported results almost entirely on the temporal category—an external check that corroborates W2: EverMemOS: its own paper ([4] Table 2, GPT-4.1-mini) reports 77.4% temporal and 89.7% knowledge-update; here (Qwen3-30B) it scores 52.5% temporal but 86.7% knowledge-update—knowledge update essentially unchanged (-3), temporal collapses (-25). Mem0: In EverMemOS's paper, 72.18% temporal in that table vs 27.8% here, its temporal drop (∼-44) roughly doubles its knowledge-update drop. One might attribute the temporal drop to the weaker backbone (Qwen3-30B vs GPT-4.1-mini). But W2 shows the failures are missing dates, not failed arithmetic—so the backbone is not the cause, and the shortfall is more-likely the configuration, not the baselines' temporal capability. References: [1] https://arxiv.org/abs/2501.13956; [2] https://github.com/Concyclics/MemForest/tree/main; [3] https://github.com/mem0ai/mem0/issues/3256; [4] https://arxiv.org/abs/2601.02163.*

Response. We address this discrepancy in four parts.

(1) LoCoMo reporting scope. Please refer to R1-W2. The submitted overall score included category 5, whereas the public Mem0 and EverMemOS headline runners exclude this adversarial category. The pinned Mem0 source identifies category 5 as excluded at benchmarks/locomo/prompts.py (lines 9–14) and evaluates categories 1–4 at benchmarks/locomo/prompts.py (line 33). The revised LoCoMo headline therefore uses the same 1,540-question scope. This factor applies only to LoCoMo, not LongMemEval.

(2) Timestamp propagation. As detailed in R3-W5, the timestamp mismatch occurred only in our submitted Mem0 adapter. We corrected it and reran all affected Mem0 cells; the refreshed values are in Tables 2 and 3.

(3) Judge policy. The submitted results used our stricter diagnostic prompt, while the released Mem0 LoCoMo prompt explicitly accepts dates within 14 days and durations within 50% at benchmarks/locomo/prompts.py (line 228). The Appendix A.3, Figure A1 (paired strict/public judge-sensitivity figure) freezes retrieval and answers and changes only the judge. On LoCoMo temporal, the public prompt changes EverMemOS, MemForest, and Mem0 by +17.33/+16.00/+5.33 points; on LongMemEval temporal, the changes are +4.92/+2.46/+4.92 points. The ordering is unchanged under both prompts on both temporal samples. Exact numerators and denominators are released at reproducibility/results/judge_prompt_sensitivity/stratified_summary.csv.

(4) Aligned results. The revised main tables use one fixed `deepseek-v4-flash` judge, the corresponding released public prompt, and LoCoMo categories 1–4. Among our three tested backbones, Gemma is the closest comparison to EverMemOS's published GPT-4.1-mini scores, although it is not a same-backbone reproduction [13]. On LoCoMo, EverMemOS reaches 88.57% versus the published 93.05%, leaving 4.48 points; the submitted displayed gap was 23.45 points (69.60% versus 93.05%). On LongMemEval, the revised Gemma temporal and knowledge-update scores are 69.17% and 84.62%, respectively, 8.27 and 5.12 points below the published 77.44% and 89.74%. After aligning the reporting scope, timestamp propagation, and judge policy, the remaining cross-paper gap can be largely explained by differences in backbone and judge models, together with stochastic variation in LLM generation. Because the published system was not rerun under an identical model stack, we do not attempt to decompose the individual contribution of these factors.

DETAILED RESPONSE TO REVIEWER #4

R4-D1. *The O(log N) complexity claim for MemForest assumes a balanced tree, but the paper never proves or empirically verifies that the tree remains balanced under continuous real-world writes, which can be highly skewed toward certain entities or scopes.*

Response. Please refer to R3-W2. We address the premise empirically and algorithmically. First, the Appendix D.2 (tree-height and frozen-index scaling audit) covers 250,118 trees from two complete Qwen builds, observes maximum height four, and finds no violation of the bounded-fan-out height check. Revised Sections 4.3 and 6.1 then state:

> *Normal temporal updates append time-anchored leaves, including superseded states, and split overflowing nodes upward to preserve bounded fan-out.*

> *Explicit source retraction is handled by the deletion path.*

Thus the $O(\log_k N)$ statement concerns structural insertion and dirty-path dependency depth under this maintained invariant, not end-to-end construction. The 250,118-tree audit is implementation evidence on the evaluated workloads, not a proof for arbitrary write sequences.

R4-D2. *The "temporal scope" abstraction is central to the paper but is never formally evaluated in isolation—there is no experiment showing that scope assignment is accurate or consistent.*

Response. Please refer to R3-W1 and R3-W3. Revised Section 7 classifies the evaluated temporal designs as timestamped logs, recent-memory plus background consolidation, validity intervals, and temporal graphs. Revised Section 6.2 then evaluates MemForest's temporal scopes in isolation through reviewed entity-routing, fragmentation, scene-threshold stability, and tree-family ablations. The routing audit finds 124/127 (97.6%) active entity assignments semantically valid, while the tree-family ablation separately measures the contribution of entity, scene, and session scopes. The Appendix B.4 (routing and fragmentation diagnostics) provides the labels and complete measurements.

R4-D3. *The paper does not consider whether a simple unified index that combines a scratch pad for the k-recent sessions plus a time-sensitive index that is constructed in the background (using techniques similar to dreaming from Claude) might be sufficient to address write overheads and ensure that recent sessions are immediately querable.*

Response. Please refer to R3-W1 and R3-W3. Revised Section 6.2 includes both Log-Time and Scratchpad+Background under shared canonical facts, timestamps, embeddings, and retrieval budgets. The native end-to-end tables also include MemPalace, which stores timestamped raw sessions and applies semantic top-k retrieval without write-time LLM extraction or state rewriting. These experiments cover both the reviewer's recent-plus-background design and its lightweight session-index operating point.

R4-D4. *The three tree families are described as complementary, but the paper never explains how scope conflicts are resolved when the same fact is relevant to multiple entities or scenes, and the routing logic discussion is too brief to evaluate this.*

Response. Revised Section 4.1 states:

> *A fact may enter multiple scopes; multi-assignment is not resolved by forcing one winning tree.*

Section 6.2 and Table 6 report the routing and tree-family diagnostics; please also refer to R3-W3.

R4-D5. *Scene trees rely on clustering to group semantically coherent situations, but the actual method is never described in sufficient detail to evaluate whether scene definitions are stable or meaningful.*

Response. Revised Section 4.1 specifies scene assignment and merge thresholds. Section 6.2 reports threshold stability and motivates redundant multi-scope placement. Please refer to R3-W3.

R4-D6. *The chunk size parameter is inadequately motivated— beyond the reported sweep in Appendix C, the paper offers no rule of thumb for practitioners, nor does it discuss whether different agentic tasks or types of interactions/sessions require different batching strategies.*

Response. Please refer to R1-W1 and revised Section 6.1. Figure 8(a–b) jointly reports Gold Coverage, facts/s, tokens/fact, extracted-fact count, and total tokens. The paper gives the following practical rule:

> *We select $b = 2$ because it preserves 100% Gold Coverage, remains near peak throughput, and uses fewer tokens per fact than $b = 1$. In deployment, the largest chunk that preserves fidelity and exposes enough parallel requests provides the practical control.*

R4-D7. *The cost argument for parallel extraction over a single LLM pass lacks nuance. For shorter sessions where the context fits comfortably in one pass, there is likely a tradeoff with prefill complexity that is never analyzed.*

Response. Please refer to R1-W1 and revised Section 4.1, which state the short-session and serving-saturation boundary.

R4-D8. *Canonicalization is described at a high level (normalizing surface forms, merging duplicates) but the paper provides no detail on how it is implemented—whether LLM-based, rule-based, or embedding-based—nor what prompts or consistency guarantees are used. Critically, when two parallel chunks produce contradictory facts, it is unclear how the system resolves the conflict.*

Response. Revised Section 4.1 states:

> *Canonicalization uses normalized exact matching, embedding-retrieved candidates, and a bounded LLM equivalence check. Equivalent records merge provenance and occurrence times; contradictory or otherwise non-equivalent updates remain separate facts with time anchors.*

The public implementation is src/extraction/dedup.py (lines 85–129) for the bounded LLM equivalence check and lines 158–220 for normalized/embedding candidate filtering. Its prompt is src/prompt/dedup_prompt.py (lines 11–34). Evaluated thresholds are recorded in reproducibility/manifests/runtime_configs.json (lines 11–23).

R4-D9. *The paper never discusses what happens when LLM extraction produces hallucinated or factually incorrect facts—there is no noise or error propagation analysis.*

Response. We agree. Revised Section 4.1 states:

> *Canonicalization does not verify source factuality, but retained provenance supports targeted correction.*

An undetected extraction error may be routed into multiple scopes and reflected in their derived summaries. Each canonical fact retains source provenance, so a detected error can be removed through source-linked deletion and only the affected paths and summaries need to be refreshed. This bounds repair work after detection, but it does not prevent hallucinated facts from entering memory. Section 8 identifies automatic factuality verification as future work. The Gold-Coverage diagnostic in Section 6.1 measures omission rather than hallucination and is not presented as an error-propagation analysis.

R4-D10. *The similarity or distance function used during retrieval is never specified (cosine, dot product, L2), nor is its accuracy analyzed.*

Response. Revised Section 4.2 states:

> *All similarities are cosine similarities over normalized Qwen3 embeddings. The Fact Manager, RootIndex, and NodeIndex use FAISS* `IndexFlatIP` *exact search.*

The equal-budget coverage control in Section 6.2 reports the resulting evidence access. The exact normalized inner-product implementation (equivalent to cosine) is at src/build/node_index.py (lines 113–121) and lines 182–202.

R4-D11. *The retrieval implementation is underspecified—it is unclear whether FAISS or another ANN library is used, what index structure backs it, and how the tree union is computed in practice.*

Response. Revised Section 4.2 specifies the persisted Fact Manager, RootIndex, and NodeIndex, top-k root recall, fact-to-tree union, deduplication, and branch descent. It states:

> *All similarities are cosine similarities over normalized Qwen3 embeddings. The Fact Manager, RootIndex, and NodeIndex use FAISS* `IndexFlatIP` *exact search.*

The Appendix D.2 (frozen-index scaling study) reports index sizes and measured retrieval latency. The released implementation uses src/extraction/fact_manager.py (lines 65–76) for the exact fact index and lines 500–522 for its search; src/query/retriever.py (lines 69–108) implements root recall and candidate union; and src/build/node_index.py (lines 137–202) implements index persistence and exact node search.

R4-D12. *The term "promising child node" is used in the tree browse description without ever being formally defined. In embedding-only mode it presumably means highest similarity, but in LLM-guided mode the selection criterion is never stated.*

Response. Revised Section 4.2 states:

> *In embedding-only mode, a promising child is one of the highest cosine-scoring child summaries. In LLM-guided mode, the browser presents child summaries and a tree-specific query to the branch-selection prompt, which selects the next child identifiers.*

The two branch-selection paths are at src/query/browser.py (lines 95–162) and lines 218–262.

R4-D13. *The decision criteria for merge operations are never explained—the paper does not specify what triggers a merge, what the decision logic is, or what the downstream impact on retrieval recall is.*

Response. Revised Section 4.3 states:

> *Canonical-fact merge requires semantic equivalence and combines provenance and occurrence times; non-equivalent state updates remain distinct. Tree rebalance follows the bounded-fan-out invariant.*

This separates canonical-fact merge from tree rebalance and memory migration. Revised Section 5.6 explicitly limits the measured claim:

> *Both strategies are evaluated on memory scale and on the wall-clock cost of producing the merged state; the question-answering experiments in this paper are not replayed against the merged state, so we do not report post-merge answer accuracy here.*

We therefore claim measured merge cost and state scale, not post-merge recall preservation. The corresponding source paths are src/extraction/dedup.py (lines 158–237), src/build/tree.py (lines 236–321), and src/forest/forest_merge.py (lines 246–420).

R4-D14. *The evaluation uses only two Qwen3 model variants. It is unclear whether results generalize to other LLM families (e.g., Llama, Mistral), making the improvements potentially model-family-specific.*

Response. Revised Section 5.1 states:

> *We evaluate three generative backbones: Qwen3-4B, Qwen3-30B-A3B, and Gemma-4-12B-IT [33], with* `Qwen3-Embedding-0.6B` *fixed for retrieval [37, 41].*

Tables 2 and 3 report the full cross-family results. Revised Section 5.2 states:

> *Planner is the stronger overall mode in both Qwen settings, whereas Embed is 0.60 points higher under Gemma, showing that browse-time refinement remains backbone-dependent.*

R4-D15. *Not all baselines discussed in related work are included consistently across all experiments—for example, MemPalace is absent from the migration experiment and the retrieval ablation despite being the closest design point.*

Response. Please refer to R4-D3. MemPalace remains in every end-to-end answer-quality table. Its evaluated lightweight path stores raw sessions and uses semantic retrieval without LLM extraction, summarization, or a separate maintenance phase; it therefore has no comparable materialized-state migration primitive or LLM maintenance cost to report. In the representation-controlled ablation, timestamped Flat Log is the shared-fact counterpart of this fidelity-first history, while native MemPalace remains in the main tables.

R4-D16. *There is no latency analysis under increasing memory size. The paper does not analyze how query-time latency scales as the number of MemTrees grows, which is critical for long-horizon deployments.*

Response. Revised Section 6.1 states:

> *Figure 7(b) isolates structure/routing, LLM summary refresh, and root embedding. Increasing one tree from 50 to 1,200 facts (24×) raises this time from 6.9 to 46.0 seconds (6.7×); extraction is excluded and remains linear in incoming content.*

The Appendix D.2 (frozen-index scaling study) separately reports the complete sweep. Revised Section 5.5 states:

> *A frozen-index scaling test composes 1, 2, 4, and 8 forests, increasing state from 1,428 facts and 250 trees to 10,781 facts and 1,868 trees. Under fixed top-10 Embed retrieval, p95 remains 41.79–44.13 ms; this microbenchmark excludes Planner, answer generation, construction, and loading.*

MemForest: An Efficient Agent Memory System with Hierarchical Temporal Indexing

Han Chen
National University of Singapore
Singapore, Singapore
chenhan@u.nus.edu

Zining Zhang
National University of Singapore
Singapore, Singapore
zzn@nus.edu.sg

Wenqi Pei
National University of Singapore
Singapore, Singapore
wenqi_pei@u.nus.edu

Bingsheng He
National University of Singapore
Singapore, Singapore
dcsheb@nus.edu.sg

Ming Wu
Zero Gravity Labs
United States
ming@0g.ai

Jason Zeng
Zero Gravity Labs
United States
jason@0g.ai

Michael Heinrich
Zero Gravity Labs
United States
michael@0g.ai

Wei Wu
Zero Gravity Labs
United States
wei@0g.ai

Hongbao Zhang
Zero Gravity Labs
United States
peter@0g.ai

ABSTRACT

Memory is a fundamental component for enabling long-context LLM agents, supporting persistent state across interactions through a continuous serve-and-update lifecycle. We present **MemForest**, a memory framework that reformulates agent memory as a write-efficient temporal data management problem. MemForest breaks the sequential bottleneck via parallel chunk extraction, decoupling memory construction into concurrent, independent operations. To further eliminate coarse-grained maintenance, we introduce *MemTree*, a hierarchical temporal index that organizes memory as time-ordered trees rather than flat global summaries. This design replaces full-state rewrites with localized per-node updates, reducing maintenance cost to the affected tree paths while preserving temporally evolving states. We evaluate MemForest on LongMemEval-S and LoCoMo. With Qwen3-30B, MemForest reaches 81.8% pass@1 on LongMemEval-S and 84.09% on LoCoMo categories 1–4, while achieving build rates 6.0× and 9.5× those of EverMemOS in our measured LongMemEval-S and LoCoMo workloads, respectively.

KEYWORDS

LLM agents, agent memory, persistent memory, temporal indexing, hierarchical retrieval, write-efficient memory

PVLDB Reference Format:
Han Chen, Zining Zhang, Wenqi Pei, Bingsheng He, Ming Wu, Jason Zeng, Michael Heinrich, Wei Wu, and Hongbao Zhang. MemForest: An Efficient Agent Memory System with Hierarchical Temporal Indexing. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/Concyclics/MemForest.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Large language model (LLM) agents are increasingly expected to sustain personalized and stateful behavior across interactions that span days, weeks, or months [24, 26, 43]. This requirement arises in applications such as conversational assistants, long-lived task agents, and interactive social agents, where useful behavior depends on preserving user preferences, prior commitments, and accumulated experiences over time. This, in turn, requires an efficient and effective memory system that transforms interaction streams into a structured memory state that remains useful as evidence accumulates and user state evolves. Recent memory systems have made substantial progress in managing long-context interactions through hierarchical structures, online/offline consolidation, and temporal graphs [3, 9, 13, 17, 27]. At the same time, recent database systems work has begun to treat LLM+retrieval workloads as first-class data systems, optimizing retrieval–inference pipelining, cache reuse, and persistent vector infrastructures for LLM applications [1, 14, 31, 39]. In this paper, we focus on improving the efficiency and effectiveness of persistent agent memory systems under this systems perspective.

Current memory systems can be viewed as having three core functions: *extraction*, which converts raw interactions into persistent memory records; *retrieval*, which fetches relevant context for downstream response generation; and *maintenance*, which updates, consolidates, and restructures existing knowledge over time [9, 42]. In continuous workloads, the placement and dependencies of these stages determine how quickly new evidence becomes queryable [3, 9, 13, 17]. Figure 1 summarizes the resulting build-rate–accuracy operating points.

Two dependency patterns recur. First, LLM-based extraction and maintenance can form state-dependent request chains before new evidence becomes queryable; EverMemOS, for example, performs streaming event extraction followed by consolidation [13]. Second, some profile- or summary-centric paths periodically rewrite compact hot states such as user profiles or global summaries [17, 24]. Even when only minor new evidence arrives, such a path must reread and rewrite the maintained object. As memory accumulates, this imposes a maintenance cost and latency floor that scales with maintained-state size rather than with newly arrived evidence.

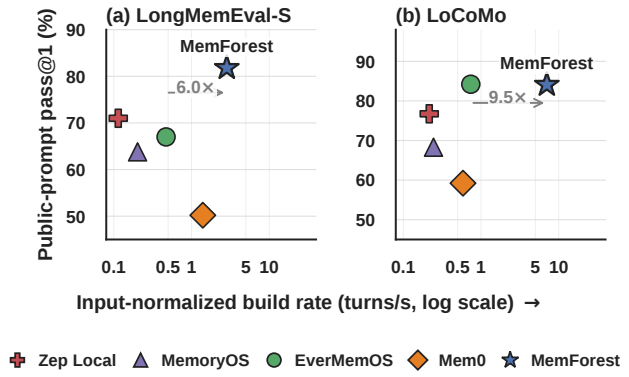

Figure 1: Qwen3-30B accuracy and memory-build rate on LongMemEval-S and LoCoMo categories 1–4.

However, improving write efficiency alone is not sufficient, because long-context agent memory is inherently temporal [11, 20, 27, 34]. User states evolve, facts are revised, and older information often remains necessary for complex reasoning. For example, if a user first lived in Boston, later moved to New York, and then relocated to San Francisco, a memory system should support not only the current-state query of where the user lives now, but also historical and transition queries such as where the user lived before New York and when the move occurred. We therefore frame long-context agent memory as a write-efficient temporal data management problem, in which persistent memory must remain incrementally maintainable while preserving historical state evolution [2, 8]. The core challenge is to overcome the trade-off between write efficiency and faithful temporal memory representation: a system must minimize the serial delays of *extraction* and the state-size-dependent costs of *maintenance* in order to maximize update throughput, while rigorously preserving historical states for long-context reasoning in order to maximize answer accuracy.

In this paper, we propose **MemForest**, a memory architecture designed around this write-efficient temporal objective. MemForest combines parallel chunk extraction, canonical fact consolidation, and *MemTree*—a hierarchical temporal index that materializes scoped memory as time-ordered trees rather than flat records or repeatedly rewritten profiles. MemForest adopts a similar high-level intuition to write-optimized indexing in database systems, where update costs are reduced by avoiding repeated rewrites of compact indexed state, as exemplified by LSM-trees [23], although the maintained object here is persistent agent memory rather than key-value state. Parallel chunk extraction dismantles the serial *extraction* bottleneck by decoupling and processing new interactions concurrently. Canonical fact consolidation repairs the semantic fragmentation inherently introduced by such parallelization. To optimize *maintenance*, MemTree replaces full-state rewrites with localized per-node updates and lazy summary regeneration, so that index-maintenance cost scales with the affected tree paths and distinct dirty nodes rather than with the total accumulated memory size (Section 4.4). At query time, MemForest leverages this structure to perform coarse-to-fine *retrieval*, navigating from broad interval summaries down to precise leaf-level evidence [7, 29, 30].

We evaluate MemForest on two long-context memory benchmarks, LongMemEval-S and LoCoMo [20, 34]. With Qwen3-30B, MemForest reaches 81.8% pass@1 on LongMemEval-S and 84.09% on LoCoMo categories 1–4. Its build rate is 6.0× and 9.5× that of EverMemOS in our measured LongMemEval-S and LoCoMo workloads, respectively. This efficiency matters because, in long-horizon deployments, *extraction* and *maintenance* costs are paid repeatedly as new sessions arrive rather than once as offline preprocessing. Overall, MemForest improves the memory substrate by accelerating write operations, explicitly preserving temporal state evolution, and exposing retrieval across multiple granularities.

Our contributions are threefold:

- We characterize how LLM-in-the-loop extraction and state-size-dependent profile or summary maintenance determine the write critical path of long-context memory systems.
- We introduce MemForest, a memory architecture that resolves these bottlenecks by combining parallel extraction with hierarchical temporal indexing, enabling localized updates, variable-granularity retrieval, and a persistent, queryable, and temporally evolving memory substrate under continuous writes.
- We show that this architectural shift improves the speed–accuracy trade-off on LongMemEval-S, where MemForest is the strongest overall system among the evaluated stateful baselines, while remaining competitive on LoCoMo with substantially reduced write-path cost across both benchmarks.

The remainder of this paper is organized as follows. Section 2 introduces the workload model and problem formulation for long-context agent memory. Section 3 presents the system overview of MemForest and its core structure, MemTree. Section 4 provides the design and implementation details of its extraction, retrieval, and maintenance workflows. Section 5 evaluates MemForest on LongMemEval-S and LoCoMo. Section 6 provides ablation studies and analysis of the key design choices. We review related work in Section 7, and conclude this paper in Section 8.

2 BACKGROUND AND PROBLEM FORMULATION

2.1 Workload Model

We consider a long-lived agent that interacts with a user through a sequence of sessions

$$\mathcal{D} = (S_1, S_2, \ldots, S_T), \tag{1}$$

where each session S_t is a multi-turn interaction arriving over time. The system must continuously extract new sessions, maintain memory across the full interaction horizon, and answer future queries against the accumulated memory state. This workload has three key properties. First, new sessions should become queryable without requiring full reconstruction of prior memory. Second, once written, memory may be queried many times by downstream tasks, including current-state lookup, multi-session recall, knowledge updates, and temporal reasoning. Third, practical systems may need to support merge, deletion, or migration as memory policies evolve

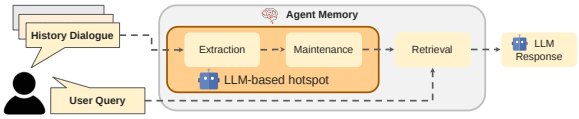

Figure 2: A generic workflow of agent memory systems. New dialogue history first goes through *extraction*, then *maintenance* updates the existing memory state, and future user queries trigger *retrieval* over the maintained memory. In many existing systems, the dominant LLM hotspot lies on the write path of extraction and maintenance.

over time [13, 19, 27, 32]. This setting is reflected in long-context benchmarks such as LoCoMo and LongMemEval [20, 34].

Existing memory systems typically consist of three stages: *extraction*, which converts newly arrived interactions into memory records; *maintenance*, which updates, merges, reorganizes, or refreshes existing memory state; and *retrieval*, which recalls relevant memory for downstream response generation [9, 17, 42]. Figure 2 illustrates this common workflow. Unlike one-shot long-context prompting, memory construction is therefore not a one-time preprocessing step, but part of an ongoing mixed read-write workload.

2.2 Write-Path and Temporal Design Trade-offs

We compare prior systems along two dimensions that shape continuous memory: dependencies on the online write path and the representation used to preserve temporal evidence.

2.2.1 Write-Path Dependencies. Representative systems combine different persistent representations with different write workflows. Table 1 separates these two design dimensions. LLM work enters the write path through event extraction and consolidation in EverMemOS, extraction and compression in LightMem, graph extraction and linking in Zep, or updates over retrieved facts and tiered state in Mem0 and MemoryOS [3, 9, 13, 17, 27]. MemPalace instead indexes raw history and defers abstraction to retrieval [22]. These choices move work among construction, maintenance, and query-time reconstruction.

2.2.2 Issues in Temporal Memory Systems. Temporal memory adds a second requirement [11, 20, 27, 34]. Long-context memory must preserve not only what is true now, but also what used to be true and when transitions occurred. For example, if a user lived in Boston, later moved to New York, and then relocated to San Francisco, the system should support current-state, historical-state, and transition queries over the same trajectory.

Existing baselines preserve temporal evidence through concrete, different mechanisms. EverMemOS keeps relative time expressions together with resolved absolute dates in episodic and event memories, then uses LLM-guided multi-round retrieval [13]. MemPalace indexes timestamped raw sessions, while LightMem and MemoryOS separate recent from consolidated or tiered memory [9, 17, 22]. Zep/Graphiti maintains bi-temporal graph edges and source episodes with validity and provenance [27]. MemTree instead organizes canonical facts into scoped, time-ordered hierarchies for coarse-to-fine access. These mechanisms move temporal work among write-time extraction, state maintenance, and query-time reconstruction. Our objective is to preserve evolving state while limiting refresh to the affected access paths.

2.3 Problem Formulation

We first introduce the following concepts.

Persistent state consists of the canonical memory contents that define the durable semantics of the system. In MemForest, this includes canonical facts, fact-to-tree mappings, cluster states, session references, and the tree structures that preserve temporally organized memory within each scope.

Derived artifacts are auxiliary structures materialized from persistent state to accelerate access. In MemForest, these include interval summaries, node embeddings, and root-level index entries. They improve efficiency, but they are not the source of truth and can be selectively refreshed after writes.

Write-path latency measures the wall-clock time required to convert newly arrived interaction history into queryable memory. This is the primary systems metric for write responsiveness.

Token cost measures the total model input/output usage incurred during memory construction and maintenance. This captures model-side resource consumption even when latency is partially hidden by parallelism.

Given a continuous session stream $\mathcal{D}$, our goal is to maintain a persistent memory substrate $\mathcal{M}$ that improves the efficiency of continuous memory construction while preserving the fidelity of temporally evolving state. In particular, we seek a design whose write cost is governed by the size of the incoming update rather than the size of the entire accumulated memory, and whose systems behavior can be evaluated along two primary dimensions: write-path latency and token cost. MemForest addresses this objective through three design choices: it uses canonical facts as stable write units, separates persistent state from derived access artifacts, and materializes scoped memory as a temporal index that supports localized maintenance and coarse-to-fine retrieval.

3 MEMFOREST OVERVIEW

MemForest is a persistent temporal memory substrate for long-context agents that supports three workflows—ingestion, retrieval, and maintenance—over a shared persistent state. Figure 3 shows the end-to-end system, and Figure 4 illustrates the core temporal index.

Design rationale. MemForest is designed to satisfy three requirements of long-horizon agent memory under continuous writes. First, new sessions should be incorporated incrementally with low write-path latency, so that memory remains fresh and queryable without introducing a serialization bottleneck on the update path. Second, updates should preserve temporally evolving state rather than collapsing memory into a single latest-state summary. The resulting memory must remain queryable not only for current-state lookup, but also for historical-state and temporal reasoning queries. Third, the write path should avoid any global memory object or global all-fact maintenance step. Each update should modify only the affected region of memory, so that maintenance cost depends on the size of new evidence rather than the size of accumulated state.

Table 1: Temporal-memory designs and write-path complexity.

System	Persistent representation	Temporal accumulation	Implemented write path	LLM-wave depth
Mem0	Indexed fact store	Timestamped facts under configured ingestion metadata	LLM ADD/UPDATE/DELETE decision over retrieved candidates	$O(\lceil N/P \rceil)^* + O(N)$
MemoryOS	Short-, mid-, and long-term tiers	Recent memory with promotion and background consolidation	Session-chain update and state-dependent promotion	$O(N)$
EverMemOS	Event logs and consolidated memory	Timestamped events plus semantic consolidation	Episode/event extraction followed by consolidation	$O(N)$
LightMem	Compressed indexed memories	Recent online records plus offline/background updates	Extraction/compression with state-dependent updates	$O(\lceil N/P \rceil)^*$; impl. $O(N)$
MemPalace	Fidelity-first raw history	Timestamped raw events	Lightweight indexing; abstraction deferred to retrieval	0
Zep Local	Bi-temporal graph and episodic store	Versioned graph edges with retained provenance	Ordered episode extraction and graph maintenance	$O(N)$
MemForest (ours)	Scoped, time-ordered MemTrees	Canonical facts plus interval summaries	Parallel extraction and local dirty-path refresh	$O(\lceil N/P \rceil + \log N)$

N counts method-native input units in one complete build and P is the per-instance LLM-worker budget; * marks parallel-feasible extraction whose released add loop is serial. The column counts autoregressive request waves, not token or non-LLM work; fixed P leaves MemForest's bound $O(N)$.

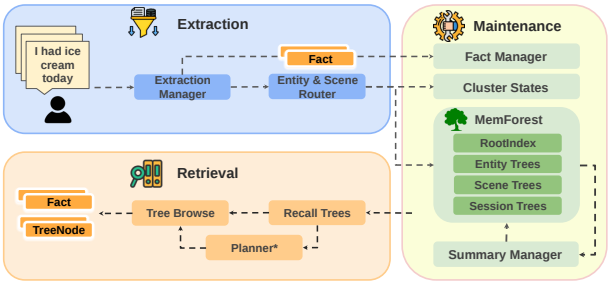

Figure 3: MemForest overview. New sessions are converted into canonical facts, materialized into scoped MemTrees, and queried through forest recall followed by tree browse. Maintenance edits only affected persistent state and selectively refreshes derived artifacts such as summaries, embeddings, and RootIndex rows. Optional modules are marked with ∗.

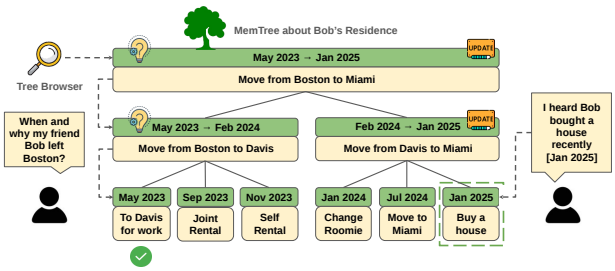

Figure 4: MemTree. Leaves preserve time-ordered evidence, internal nodes summarize contiguous intervals, and the root supports coarse tree-level recall. The same structure supports both query-time browse and localized refresh after writes.

MemForest addresses these requirements through three design choices. It uses canonical facts as stable write units, separates persistent state from derived access artifacts, and materializes scoped memory as temporal indexes that support localized maintenance and coarse-to-fine retrieval.

3.1 Shared Memory Substrate

The core abstraction of MemForest is a shared memory substrate with two layers: *persistent state* and *derived artifacts*. Persistent state stores the durable semantics of memory and serves as the source of truth. In MemForest, it includes canonical facts, fact-to-tree mappings, cluster states, and the scoped tree structures themselves. Derived artifacts are auxiliary access structures regenerated from persistent state to accelerate retrieval and maintenance, including interval summaries, node embeddings, and root-level index rows.

This separation is central to the system design. Writes modify only affected persistent regions, while dependent derived artifacts are refreshed selectively rather than through full-state rewriting. As a result, the same substrate can support incremental writes, localized maintenance, and later lifecycle operations such as merge or deletion.

3.2 MemTree as a Scoped Temporal Index

MemTree is the core storage abstraction of MemForest. Each materialized scope is represented as a balanced temporal tree whose leaves preserve fine-grained evidence in chronological order, whose internal nodes summarize contiguous time intervals, and whose root provides a coarse representation for tree-level recall. Unlike flat memory stores, MemTree preserves state evolution directly in the index: older states remain accessible as earlier leaves and intervals rather than being overwritten by a latest-state summary.

MemTree serves two roles at once. On the write path, updates affect only touched leaves and ancestor regions, enabling localized maintenance. At query time, the same structure supports coarse-to-fine retrieval, allowing search to begin at root or interval summaries and descend only where finer evidence is needed.

MemForest materializes three complementary tree families. Session trees preserve dialogue order and local fidelity. Scene trees provide broader semantic access through clustered latent topics. Entity trees provide direct access to recurring subjects that accumulate evidence over time. Together, they expose dialogue-, topic-, and entity-scoped access paths over the same persistent substrate, so that retrieval can choose the scope that best matches a query rather than being tied to a single global index.

3.3 Workflow Overview

Ingestion workflow. Ingestion avoids coupling each new session to a shared global object. Raw interaction streams are processed chunk-locally, consolidated into canonical facts, routed into session-, entity-, and scene-level scopes, and materialized into the corresponding MemTrees. Because a new session is decomposed into stable write units and inserted only into affected scopes, write cost tracks the incoming update rather than the size of accumulated memory.

Retrieval workflow. Retrieval separates coarse pruning from fine-grained evidence search so that LLM cost is spent only where it

improves answer quality. MemForest first performs forest-level recall to select a small set of candidate trees, then browses within those trees to locate answer-bearing evidence. This two-stage design keeps semantic and temporal grouping intact during coarse pruning while allowing evidence search to descend from root and interval summaries to leaf-level facts or dialogue units only where needed.

Tree browse supports two operating modes under the same retrieval stack: a lightweight embedding-only mode suitable for latency-sensitive settings, and an LLM-guided mode that supports higher-accuracy evidence localization when additional traversal cost is acceptable. In the default high-accuracy configuration, MemForest further uses a planner to generate targeted per-tree subqueries based on recalled root summaries; Section 6.2 analyzes the effect of planner use and browse policy.

Maintenance workflow. Maintenance reuses the ingestion-side locality principle so that incremental updates, merge, deletion, and related lifecycle operations remain bounded by the affected state rather than the full accumulated memory. Rather than rewriting a global memory object, MemForest edits only affected persistent regions and then selectively refreshes dependent derived artifacts, so that maintenance cost scales with touched scopes and access paths.

4 IMPLEMENTATION DETAILS

Section 3 described the overall MemForest architecture. We now present the implementation of its core mechanisms.

4.1 Extraction Workflow

The extraction workflow converts raw interaction streams into canonical facts and materializes them into scoped MemTrees through four stages: parallel extraction, canonicalization, routing, and tree materialization.

Parallel extraction. Given a session

$$D = (u_1, u_2, \ldots, u_T), \tag{2}$$

MemForest partitions it into fixed-size chunks

$$C(D) = \{c_i\}_{i=1}^{\lceil T/b \rceil}, \qquad c_i = (u_{(i-1)b+1}, \ldots, u_{\min(ib,T)}). \tag{3}$$

Chunks are processed independently up to the concurrency budget, exposing inference parallelism and avoiding the long serialized extraction path of profile-centric systems. This creates a latency–token trade-off: smaller chunks expose more parallel requests and bound each generation, but repeat prompt/schema input and JSON scaffolding; larger chunks amortize that overhead but reduce parallelism and can lose answer-bearing evidence as context grows [18]. For short sessions or below serving saturation, repeated prefill can outweigh the parallel-latency benefit.

Canonicalization. Because chunk-local extraction fragments memory, MemForest canonicalizes the outputs before indexing. Each canonical fact contains retrieval-ready fact text, temporal anchors, entity labels, and scene labels, and is stored in the Fact Manager as the system's stable write unit. Canonicalization uses normalized exact matching, embedding-retrieved candidates, and a bounded LLM equivalence check. Equivalent records merge provenance and occurrence times; contradictory or otherwise non-equivalent updates remain separate time-anchored facts. Canonicalization does not verify source factuality, but retained provenance supports targeted correction.

Routing. Canonical facts are then routed into session, entity, and scene scopes. Session scope comes directly from the source dialogue. Entity scope is induced from normalized entity labels. Scene scope is induced by clustering over canonical facts and maintained with cluster states containing centroids and member-fact identifiers. A fact may enter multiple scopes; multi-assignment is not resolved by forcing one winning tree. Scene assignment uses cosine similarity to active cluster centroids, creates a new scene below the activation threshold, and merges clusters only above the configured merge threshold. Session and scene paths remain available when an entity overlay is not activated. This stage remains lightweight and requires no additional LLM calls after extraction.

Tree materialization. Finally, routed facts are inserted into the corresponding MemTrees. Entity and scene trees use atomic facts as leaves, while session trees use original dialogue units to preserve a high-fidelity channel. Instead of rewriting a compact global state, MemForest inserts only the new evidence into affected scopes.

4.2 Retrieval Workflow

At query time, MemForest separates retrieval into *forest recall* and *tree browse*. Forest recall selects candidate trees; tree browse locates answer-bearing evidence inside them.

Forest recall. Given a query q, MemForest first recalls candidate trees from the forest. Root summaries provide a natural coarse retrieval unit, but may lose localized lexical cues. Atomic facts often preserve stronger signals for entity names, dates, and short event descriptions. MemForest therefore uses *union recall*.

First, we retrieve the top-k trees using root-summary embeddings:

$$C_{\text{root}}(q) = \text{TopK}\big(\text{sim}(e(q), e(r_T))\big)_{T \in \mathcal{F}}, \tag{4}$$

where $e(q)$ is the query embedding, $e(r_T)$ is the embedding of tree T's root summary, and $\mathcal{F}$ is the forest. All similarities are cosine similarities over normalized Qwen3 embeddings. The Fact Manager, RootIndex, and NodeIndex use FAISS `IndexFlatIP` exact search.

Second, we retrieve the top-k atomic facts,

$$R_{\text{fact}}(q) = \text{TopK}_f\big(\text{sim}(e(q), e(f))\big), \tag{5}$$

map each fact back to the trees in which it appears, and collect

$$C_{\text{fact}}(q) = \bigcup_{f \in R_{\text{fact}}(q)} \text{Trees}(f), \tag{6}$$

where $\text{Trees}(f)$ denotes the set of trees containing fact f.

We then rank trees in the union pool by

$$\text{score}(q, T) = \max\big(\text{sim}(e(q), e(r_T)),\ s_{\text{fact}}(q, T)\big), \tag{7}$$

where

$$s_{\text{fact}}(q, T) = \begin{cases} \max\limits_{f \in T \cap R_{\text{fact}}(q)} \text{sim}(e(q), e(f)) & \text{if } T \cap R_{\text{fact}}(q) \neq \emptyset, \\ -\infty & \text{otherwise.} \end{cases} \tag{8}$$

The final recalled tree set is

$$C(q) = \text{TopK}_{T \in C_{\text{root}}(q) \cup C_{\text{fact}}(q)} \ \text{score}(q, T). \tag{9}$$

Root recall captures scope-level relevance, while fact-to-tree recall recovers trees whose relevance is concentrated in local evidence.

Tree browse. MemForest then browses within the recalled trees by descending from root and interval summaries to finer-grained branches until reaching leaf evidence. The retrieved leaves are resolved back to canonical facts or dialogue units, reranked, and assembled into the final answer context.

This browse stage supports two modes. In embedding-only mode, a promising child is one of the highest cosine-scoring child summaries. In LLM-guided mode, the browser presents child summaries and a tree-specific query to the branch-selection prompt, which selects the next child identifiers. Both modes share the same recalled trees and MemTree structure; they differ only in branch-selection policy.

Planner-guided per-tree subqueries. In the high-accuracy setting, tree browse can be paired with a planner. Given the original query and the recalled root summaries, the planner generates one targeted subquery per tree. This helps because different recalled trees often correspond to different aspects of the question, and the root summary provides a compact tree-level description.

The planner is therefore a traversal controller rather than an answer generator. Without it, the original query q is used for all trees. With it, browse proceeds using specialized per-tree subqueries. Section 6.2 studies its effect on retrieval quality and efficiency.

4.3 Maintenance Workflow

The main systems idea of MemForest is to separate *eager structural maintenance* from *lazy semantic refresh*. Structural edits are applied immediately, while summaries and other derived artifacts are refreshed later through dirty-node batching.

Eager structural maintenance. When new evidence arrives, MemForest routes each record into its affected trees, creates new leaves, updates placement maps, and performs any required insertion, split, or rebalance. Under the bounded-fan-out balancing policy, a k-ary tree with N leaves touches only the new leaf and its ancestor path, so structural maintenance affects

$$O(\log_k N) \tag{10}$$

nodes per affected tree. Normal temporal updates append time-anchored leaves, including superseded states, and split overflowing nodes upward to preserve bounded fan-out. Explicit source retraction is handled by the deletion path. A write completes after its dirty summaries are refreshed.

Lazy semantic refresh. Within each write batch, records are grouped by touched tree and affected ancestors are marked dirty. Dirty nodes are deduplicated, grouped by level, and refreshed bottom-up before the write returns. Entity and scene leaves can pass through their canonical fact payload directly, whereas session leaves summarize raw dialogue units. Internal nodes summarize their children only after lower levels have been refreshed. Because multiple writes may share ancestors, each dirty node is summarized at most once per flush. Refresh is scheduled level-parallel, and nodes from different trees can be batched together. Algorithm 1 summarizes the process. Summary regeneration is the LLM-based maintenance step: a parent waits for its refreshed children, while same-level nodes and nodes in different trees form independent request waves.

Algorithm 1: MemTree maintenance with eager structure and lazy refresh

Input : forest state $\mathcal{M}$, incoming canonical facts $\mathcal{F}$, incoming dialogue units $\mathcal{S}$
Output : updated persistent state and refreshed derived artifacts

$\mathcal{B} \leftarrow \text{RouteAndActivate}(\mathcal{M}, \mathcal{F}, \mathcal{S})$
foreach $(T, R) \in \mathcal{B}$ **do**
  foreach $r \in \text{SortByTime}(R)$ **do**
    $\ell \leftarrow \text{CreateLeaf}(r)$
    $\text{AttachAndRebalance}(T, \ell, r.time)$
    $\text{UpdatePlacementMap}(r, T, \ell)$
    $\text{MarkDirtyAncestors}(\ell)$
$\mathcal{T}_{\text{aff}} \leftarrow \{T \mid (T, R) \in \mathcal{B}\}$
$\mathcal{D} \leftarrow \text{CollectDirtyNodesByLevel}(\mathcal{T}_{\text{aff}})$
for $\lambda \leftarrow 0$ **to** $\text{MaxLevel}(\mathcal{D})$ **do**
  foreach $u \in \mathcal{D}[\lambda]$ ***in parallel*** **do**
    if $\lambda = 0 \wedge \text{TreeType}(u) \in \{\textsc{entity}, \textsc{scene}\}$ **then**
      $u.summary \leftarrow \text{Passthrough}(u.payload)$
    else if $\lambda = 0$ **then**
      $u.summary \leftarrow \text{SummarizeCellText}(u.payload)$
    else
      $u.summary \leftarrow \text{SummarizeChildren}(u.children)$
foreach $u \in \text{AffectedNonLeafNodes}(\mathcal{T}_{\text{aff}})$ ***in parallel*** **do**
  $u.embedding \leftarrow \text{Embed}(u.summary)$
$\text{RefreshRootRows}(\mathcal{M}, \mathcal{T}_{\text{aff}})$
return $\mathcal{M}$

Lifecycle operations. The same local-update principle extends to other lifecycle operations.

Merge. Canonical-fact merge requires semantic equivalence and combines provenance and occurrence times; non-equivalent state updates remain distinct. Tree rebalance follows the bounded-fan-out invariant. MemForest then reconciles cluster states and the corresponding trees. If two scopes align to the same merged tree, only their touched subtrees are merged; otherwise unmatched trees can be copied directly.

Delete. Normal state evolution retains superseded facts. If source dialogue is explicitly retracted, MemForest removes the corresponding facts and tree nodes, locally rebalances affected trees, updates the mappings, and refreshes only invalidated summaries and embeddings.

Migration. External memory can be imported as another materialized forest and integrated through the same merge path. Compatible scopes reuse the canonical-fact and subtree merge procedures above, unmatched trees are copied directly, and only affected summaries and root-index entries are refreshed. Migration therefore reuses constructed state instead of replaying the source dialogues.

4.4 Cost Rationale and Update Locality

MemForest shares the high-level intuition of write-optimized structures such as LSM-trees [23]: expensive maintenance is deferred and batched rather than triggered as repeated full-state rewrites. The deferred work here is semantic refresh, and we next quantify how this keeps maintenance cost bound to local updates.

Let F be the number of incoming canonical facts, U the maximum number of trees touched by one fact, N the maximum number of leaves in a touched tree, and k the branching factor. Across the batch, structural maintenance touches one root-to-leaf path per fact–tree assignment:

$$T_{\text{struct}} = O(FU \log_k N). \tag{11}$$

Across a batch, let D be the number of distinct dirty nodes after ancestor deduplication. Summary regeneration depends on D, not on total memory size:

$$T_{\text{refresh}} \propto \sum_{\lambda=0}^{h} |D_\lambda|, \qquad D = \bigcup_{\lambda=0}^{h} D_\lambda, \tag{12}$$

where D_λ is the dirty-node set at level λ and $h = \lceil \log_k N \rceil$. Thus total refresh work is $O(D)$ and a touched tree has at most $O(\log_k N)$ dependency levels. With an LLM worker budget P, the actual number of refresh request waves is $\sum_{\lambda=0}^{h} \lceil |D_\lambda|/P \rceil$, as analyzed in Section 5.4.

The key claim is therefore locality: semantic maintenance is bounded by affected paths and distinct dirty nodes rather than by the full accumulated memory state. Under bounded fan-out and balanced MemTrees, structural insertion and dirty-path dependency depth are $O(\log_k N)$; extraction scales with incoming content, and refresh work scales with D.

4.5 Tree-Level Concurrency and Snapshot Semantics

Each MemTree owns an independent reader–writer lock. Insertion and summary refresh take the touched tree's write lock, so distinct trees proceed concurrently while same-tree writes serialize. Range queries, browse, and snapshot export use shared read locks; fact-store and root-index updates use separate component locks. The online `auto_update` path refreshes dirty summaries and root vectors before returning, and snapshot save atomically renames a completed temporary directory into place. Detailed cross-session conflict measurements are reported in Appendix D.3.

5 EVALUATION

We now evaluate MemForest as an end-to-end long-context memory system. Throughout this section, we use *write path* to refer to the extraction and maintenance pipeline that transforms incoming dialogue history into queryable persistent memory. Our evaluation studies three questions: (1) how effective MemForest is on long-context conversational memory benchmarks, (2) how the two retrieval operating modes of MemForest compare in answer quality, and (3) how much the system reduces write-path and query-time cost relative to representative baselines.

5.1 Experimental Setup

All systems first transform each benchmark history into persistent memory through their native write paths. **MemForest-Planner** uses union recall and planner-guided tree browsing; **MemForest-Embed** uses the same memory but embedding-only descent.

We evaluate three generative backbones: Qwen3-4B, Qwen3-30B-A3B, and Gemma-4-12B-IT [33], with `Qwen3-Embedding-0.6B` fixed for retrieval [37, 41]. Models are served on dedicated H100 GPUs using vLLM 0.18.0 and FlashAttention [4].

LongMemEval-S contains 500 question-specific instances across six categories; LoCoMo contains 1,986 questions across single-hop, multi-hop, open-ended, temporal, and adversarial categories [20, 34]. We compare EverMemOS [13], LightMem [9], MemoryOS [17], MemPalace [22], and Mem0 [3]. Following Zhou et al. [44], Zep Local is implemented with the open Graphiti temporal-graph core, Neo4j, and self-hosted generation and embedding services. It represents the local Graphiti design, not the Zep Cloud service.

We report pass@1, native write rate, and query latency. Final answers are evaluated by `deepseek-v4-flash` [35] at temperature zero using each benchmark's public judge prompt. LongMemEval-S uses all 500 questions; the primary LoCoMo result uses categories 1–4 (1,540 questions), while adversarial category 5 is reported separately in Appendix A.4. Mem0 receives source event timestamps, ingests LoCoMo histories per session with complete message coverage, and uses a global top-10 flat budget.

We preserve each baseline's native construction, retrieval, and answer interface. Flat systems use a final top-10 fact budget. MemForest retrieves ten native tree units before expansion, MemPalace retrieves ten sessions, and Zep uses its native per-evidence-class limit for heterogeneous graph objects. Equal-flat-fact retrieval is evaluated separately in Section 6.2. Configurations and validation records are available in the baseline guide and Appendix A.2.

5.2 Main Result on LongMemEval

Table 2 reports LongMemEval-S pass@1. Across all three backbones, a MemForest variant achieves the highest overall accuracy. MemForest-Planner reaches 72.60% and 81.80% under Qwen3-4B and Qwen3-30B; under Gemma, MemForest-Embed reaches 78.40%, 0.40 points above EverMemOS. A MemForest variant also has the highest temporal score under each backbone (tied between its two modes at Qwen3-4B). Planner is the stronger overall mode in both Qwen settings, whereas Embed is 0.60 points higher under Gemma, showing that browse-time refinement remains backbone-dependent. Section 5.4 compares this answer quality with memory-build rate.

5.3 Main Result on LoCoMo

Table 3 reports categories 1–4. Under Qwen3-4B, MemForest-Planner reaches 78.12%, 1.10 points below EverMemOS; under Qwen3-30B, the two are effectively tied at 84.09% and 84.22%. A MemForest variant leads multi-hop and open-ended under both Qwen backbones, while Planner is within 0.62 and 0.31 temporal points of EverMemOS, respectively.

Under Gemma, EverMemOS leads overall by 5.91 points, but its temporal margin over MemForest-Planner is only 0.31 points. Thus the larger overall difference is category- and backbone-dependent

Table 2: LongMemEval-S pass@1. SS = single-session; K-Upd. = knowledge update.

Method	Overall	SS-User	SS-Pref.	SS-Asst.	K-Upd.	Multi	Temp.
Qwen3-4B-Instruct-2507							
MemForest-Planner	**72.60**	**95.71**	70.00	83.93	70.51	59.40	**70.68**
MemForest-Embed	69.40	88.57	53.33	83.93	60.26	60.90	**70.68**
EverMemOS	61.20	84.29	66.67	69.64	64.10	55.64	48.12
LightMem	68.80	94.29	**76.67**	35.71	**79.49**	**65.41**	64.66
MemoryOS	64.20	87.14	46.67	89.29	50.00	57.14	60.90
MemPalace	60.00	91.43	36.67	33.93	65.38	63.91	52.63
Mem0	44.80	82.86	33.33	26.79	43.59	42.11	38.35
Zep Local	69.60	94.29	53.33	**98.21**	75.64	59.40	54.89
Qwen3-30B-A3B-Instruct-2507							
MemForest-Planner	**81.80**	**97.14**	83.33	83.93	75.64	**76.69**	**81.20**
MemForest-Embed	78.40	94.29	80.00	83.93	69.23	72.93	78.20
EverMemOS	67.00	91.43	53.33	75.00	79.49	62.41	51.13
LightMem	70.20	95.71	**86.67**	37.50	79.49	64.66	66.92
MemoryOS	63.80	87.14	60.00	89.29	62.82	52.63	53.38
MemPalace	53.60	81.43	50.00	35.71	62.82	52.63	42.86
Mem0	50.20	91.43	46.67	26.79	60.26	42.86	40.60
Zep Local	71.00	95.71	63.33	**96.43**	**82.05**	56.39	57.14
Gemma-4-12B-IT							
MemForest-Planner	77.80	95.71	63.33	46.43	82.05	74.44	85.71
MemForest-Embed	**78.40**	94.29	60.00	46.43	84.62	74.44	**87.97**
EverMemOS	78.00	95.71	83.33	76.79	84.62	72.93	69.17
LightMem	77.00	**97.14**	**86.67**	37.50	80.77	**75.19**	80.45
MemoryOS	66.00	95.71	70.00	94.64	66.67	47.37	55.64
MemPalace	72.80	88.57	50.00	94.64	79.49	60.90	68.42
Mem0	59.20	91.43	50.00	23.21	74.36	54.89	54.89
Zep Local	75.60	94.29	66.67	**96.43**	**89.74**	63.16	63.16

Table 3: LoCoMo categories 1–4 pass@1.

Method	Overall	Single-Hop	Multi-Hop	Open-Ended	Temporal
Qwen3-4B-Instruct-2507					
MemForest-Planner	78.12	80.86	81.21	53.12	75.70
MemForest-Embed	76.62	79.43	**81.56**	**56.25**	71.03
EverMemOS	**79.22**	**83.71**	79.79	47.92	**76.32**
LightMem	66.17	69.68	68.79	40.62	62.31
MemoryOS	65.45	71.70	71.99	39.58	51.09
MemPalace	51.56	55.77	68.79	29.17	32.09
Mem0	47.86	45.66	52.84	39.58	51.71
Zep Local	68.70	78.36	62.77	34.38	58.88
Qwen3-30B-A3B-Instruct-2507					
MemForest-Planner	84.09	84.66	**86.88**	**67.71**	85.05
MemForest-Embed	80.58	82.88	85.11	59.38	76.95
EverMemOS	**84.22**	**87.63**	84.04	51.04	**85.36**
LightMem	60.52	62.07	62.06	44.79	59.81
MemoryOS	68.31	75.98	72.34	50.00	50.16
MemPalace	55.84	59.81	72.70	33.33	37.38
Mem0	59.22	58.86	61.35	58.33	58.57
Zep Local	76.75	82.52	76.95	56.25	67.60
Gemma-4-12B-IT					
MemForest-Planner	82.66	80.98	89.36	57.29	88.79
MemForest-Embed	81.69	80.38	87.23	57.29	87.54
EverMemOS	**88.57**	**90.84**	**89.72**	**63.54**	**89.10**
LightMem	78.90	81.33	79.43	50.00	80.69
MemoryOS	78.51	80.98	83.33	57.29	74.14
MemPalace	55.71	58.98	77.30	28.12	36.45
Mem0	48.38	44.71	57.09	39.58	52.96
Zep Local	69.55	74.91	68.79	26.04	69.16

Table 4: Qwen3-30B memory-build rates.

	LongMemEval-S		LoCoMo	
Method	Turns/s	Rate/Zep	Turns/s	Rate/Zep
MemForest	**2.841**	**24.9×**	**7.020**	**32.5×**
EverMemOS	0.471	4.1×	0.738	3.4×
Mem0	1.397	12.3×	0.586	2.7×
MemoryOS	0.202	1.8×	0.245	1.1×
Zep Local	0.114	1.0×	0.216	1.0×

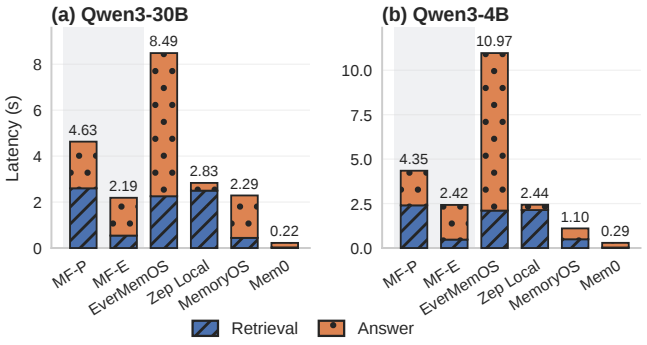

Figure 5: Retrieval and answer latency. MF-P and MF-E denote MemForest-Planner and MemForest-Embed.

rather than a uniform temporal deficit. Planner improves overall accuracy over Embed under all three backbones. Together with the LoCoMo write rate in Section 5.4, these results place MemForest on a favorable quality–efficiency operating point.

5.4 Write-Path Efficiency

Table 4 reports source turns per second from the measured Qwen3-30B build workloads. MemForest reaches 6.0× and 9.5× the EverMemOS build rate on LongMemEval-S and LoCoMo, respectively. The trace manifest records the underlying measurements.

LLM-wave critical path. Table 1 counts complete-build autoregressive request waves. For D_ℓ dirty summaries at level ℓ, $D = \sum_\ell D_\ell$, tree height H, and P workers, MemForest has

$$C_{\mathrm{MF}} = \left\lceil \frac{N}{P} \right\rceil + \sum_{\ell=0}^{H} \left\lceil \frac{D_\ell}{P} \right\rceil = O\left(\left\lceil \frac{N+D}{P} \right\rceil + H \right).$$

Bounded extraction yield and balanced bounded fan-out give $D = O(N)$ and $H = O(\log N)$, hence $O(\lceil N/P \rceil + \log N)$. This is dependency depth, not total work: fixed P gives $O(N)$, and token/non-LLM costs remain. The evaluated Mem0, MemoryOS, EverMemOS, LightMem, and Zep Local paths retain an $O(N)$ LLM chain; MemPalace has zero LLM depth but $O(N)$ indexing. At $P = 128$, Figure 7(c) shows 2.83× refresh speedup for $n = 100$ and 14.21× for $n = 4{,}000$.

Figure 8(c–d) separates extraction, canonicalization, routing, structural insertion, dirty refresh, index update, and persistence. Autoregressive extraction and dirty refresh account for 83.58% of measured build time; including hybrid canonicalization raises the LLM-involved share to 90.57%, while structural insertion, index update, and persistence together remain below 1%. Parent summaries depend on refreshed children, whereas same-level and cross-tree

calls form independent waves. A throughput-calibrated serial replay estimates 54.58 and 9.88 minutes for extraction and dirty refresh, compared with measured parallel times of 122.31 and 27.92 seconds (26.8× and 21.2× speedups). These are measured finite-resource gains. Appendix D and Appendix E report replay sensitivity, request/token statistics, and method-level write dependencies.

5.5 Query-Time Latency

We next evaluate **query-time latency**, the online cost of retrieving memory and producing an answer after memory has been constructed. This is a secondary but practical metric alongside write-path efficiency.

Figure 5 separates retrieval and answer latency under the 30B and 4B answer backbones. For Zep Local, we reuse the frozen benchmark graphs and report a serial 50-question LongMemEval sample after five warm-up questions; this excludes the queueing delay from the original high-concurrency benchmark run.

Overall latency. MemForest exposes a clear *latency–accuracy trade-off* through its two retrieval modes. The embedding-only variant achieves **2.19s** (30B) and **2.42s** (4B) total latency, comparable to MemoryOS and substantially faster than EverMemOS. The LLM-guided variant increases total latency to **4.63s** and **4.35s**, reflecting the additional reasoning cost during tree browsing.

Retrieval vs. answer cost. The breakdown shows that the main difference between the two MemForest variants lies in **retrieval time**, not answer generation. MemForest-Planner spends **2.4–2.6s** on retrieval, compared to only **0.47–0.54s** for MemForest-Embed, while answer time remains relatively stable across the two variants. This indicates that planner-guided browsing primarily adds cost on the retrieval side, while leaving the downstream answer generation stage largely unchanged.

Comparison with baselines. Compared with EverMemOS, MemForest reduces query latency substantially (e.g., **4.63s vs. 8.49s** under 30B). Part of EverMemOS's larger answer-side latency comes from its *self-judge* style answer workflow, which introduces extra reasoning overhead after retrieval. By contrast, MemForest keeps the online path simpler once evidence has been assembled. Compared with MemoryOS, MemForest-Embed achieves similar latency with higher accuracy (Sections 5.2 and 5.3), indicating a more favorable quality–latency balance. Mem0 remains the fastest system because its retrieval pipeline is minimal, but this comes with a large drop in answer quality. Zep Local requires 2.83s (30B) and 2.44s (4B) in total. Its native graph retrieval and reranking account for 2.50s and 2.13s, respectively; answer generation accounts for 0.34s and 0.30s.

Scaling. A frozen-index scaling test composes 1, 2, 4, and 8 forests, increasing state from 1,428 facts and 250 trees to 10,781 facts and 1,868 trees. Under fixed top-10 Embed retrieval, p95 remains 41.79–44.13 ms; this microbenchmark excludes Planner, answer generation, construction, and loading.

5.6 Efficient Migration

Beyond the initial write path, persistent memory must support post-construction maintenance such as migration, consolidation, and

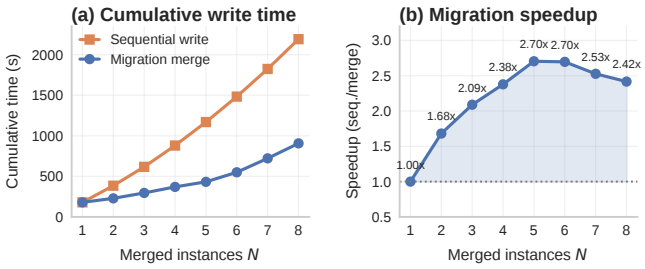

Figure 6: Sequential rewriting versus migration merge: cumulative time (left) and speedup (right).

Table 5: State size after sequential rewriting and migration merge.

N	Seq Facts	Mig Facts	Seq Trees	Mig Trees
1	1,435	1,435	249	249
2	2,692	2,716	375	405
3	4,061	4,098	562	607
4	5,540	5,590	733	792
5	6,860	6,922	895	967
6	8,101	8,174	1,044	1,128
7	9,426	9,511	1,200	1,296
8	10,705	10,801	1,362	1,472

synchronization. We build a migration workload by progressively combining one to eight independently constructed LongMemEval instances. Because the instances represent different users, this is a synthetic multi-source stress workload rather than one user's accumulating history. *Sequential write* replays every source history into a growing state, whereas *migration merge* combines already materialized MemForest states through the merge path in Section 4.3. Both strategies are evaluated on memory scale and on the wall-clock cost of producing the merged state; the question-answering experiments in this paper are not replayed against the merged state, so we do not report post-merge answer accuracy here.

Figure 6 shows that migration merge is consistently faster. It exceeds 2× speedup from N=3, peaks at 2.70× around N=5–6, and remains above 2.4× at N=8. The gain comes from reusing prior construction: migration avoids repeated LLM extraction over already processed histories, copies unaffected trees directly, and refreshes only the paths touched by cross-forest reconciliation.

Table 5 shows comparable materialized state. Across all N, fact counts differ by less than 1% and tree counts by at most about 8%; at N=8, sequential write and migration produce 10,705 versus 10,801 facts and 1,362 versus 1,472 trees. Small differences arise from stochastic extraction and summary refresh together with routing decisions during merge. Baseline adapters instead accept continued source-history writes; MemForest's merge interface targets memory transfer and synchronization scenarios such as expert memory transfer and distributed construction [12, 16, 28].

Overall, migration extends MemForest's write-efficiency benefit to post-construction maintenance by reusing already materialized memory state.

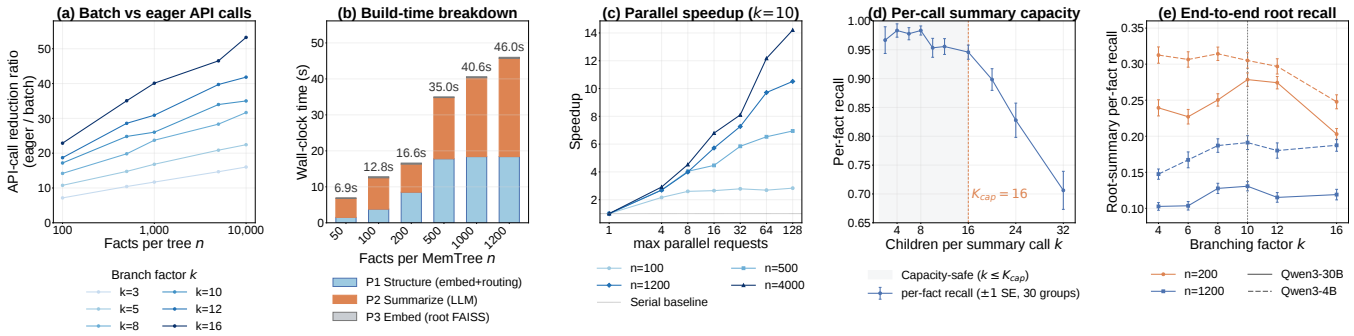

Figure 7: Write-path scalability diagnostics for MemTree. (a) Batch mark-dirty reduces LLM summary calls relative to eager per-insert rewriting. (b) MemTree build stages are profiled from 50 to 1,200 facts. (c) Level-parallel flush yields larger speedups on larger trees, showing improved scalability with data size. (d) Per-call summary capacity remains stable up to a moderate fan-out before dropping. (e) End-to-end root recall peaks at moderate branching factors, motivating the default k settings used in MemForest.

6 ABLATION STUDY

The main results show that MemForest improves both answer quality and write-path efficiency. We now ask a more targeted question: which design choices are necessary for these gains? We organize the ablations around the two central design choices of MemForest. First, we study write-path scalability, focusing on the MemTree mechanisms that make maintenance local and scalable. Second, we study retrieval accuracy on a controlled LongMemEval subset, isolating the role of temporal tree views and selective browse policies.

6.1 Write-Path Scalability

We first study the write path, since MemForest's main systems contribution is not merely faster extraction, but the ability to keep long-horizon memory maintenance local as memory grows. The key question is therefore whether MemTree maintenance remains scalable under increasing tree size, and whether moderate fan-out is necessary to balance efficiency and summary quality.

Lazy maintenance reduces redundant LLM work and improves scalability. Figure 7(a) compares eager per-insert rewriting against MemForest's batch mark-dirty update strategy. Batch refresh substantially reduces summary calls, and the reduction becomes larger as the number of facts per tree grows. This shows that lazy maintenance is not a small implementation optimization: it is what prevents repeated writes from triggering redundant LLM regeneration along overlapping ancestor paths.

The same locality benefit appears in parallel execution. Figure 7(c) shows that level-parallel flush yields larger speedups for larger trees, because bigger trees expose more same-level nodes that can be refreshed concurrently. In other words, the gain from MemTree maintenance improves with data scale rather than disappearing at larger workloads. Across 250,118 trees from two complete Qwen builds, the maximum observed height is four, with no bounded-fan-out violations. This confirms height-bounded structural paths on the evaluated writes.

Moderate fan-out is necessary for both efficiency and summary quality. Figures 7(d) and 7(e) study the branching factor k. Increasing k makes trees shallower and can reduce maintenance depth, but it also overloads each summary call. Per-call summary recall remains high up to about $k \approx 16$, after which it drops more sharply, indicating a soft summary-capacity knee. End-to-end root recall peaks at moderate k values rather than at the extremes: small k deepens the summarization chain and accumulates cascade loss, while overly large k makes each summary too flat and lossy. These results justify the use of moderate fan-out defaults in MemForest rather than treating k as an arbitrary tuning knob.

MemTree maintenance grows below linearly over the observed range. Figure 7(b) isolates structure/routing, LLM summary refresh, and root embedding. Increasing one tree from 50 to 1,200 facts (24×) raises this time from 6.9 to 46.0 seconds (6.7×); extraction is excluded and remains linear in incoming content.

Chunk size controls the latency–token trade-off. Figure 8(a–b) sweeps chunk size on an assembled long session. Larger chunks reduce total tokens chiefly by extracting fewer facts: Gold Coverage falls and tokens/fact eventually worsens, consistent with lost-in-the-middle effects [18]. We select $b = 2$ because it preserves 100% Gold Coverage, remains near peak throughput, and uses fewer tokens per fact than $b = 1$. In deployment, the largest chunk that preserves fidelity and exposes enough parallel requests provides the practical control.

6.2 Temporal Representation and Retrieval Controls

We map 319/321 official LoCoMo temporal questions to supporting facts, using 82 for selector development and 237 for held-out evaluation. All methods share facts, timestamps, embeddings, and budgets; selectors tune only on development data. Coverage is macro per-question mapped-gold recall.

Figure 8(e) shows that Log-Time leads MemTree by 0.7 points at $k = 10$; they are within 0.1 points at $k = 30$, and MemTree leads at $k = 50$ (92.0%). This places MemTree as a high-recall option at

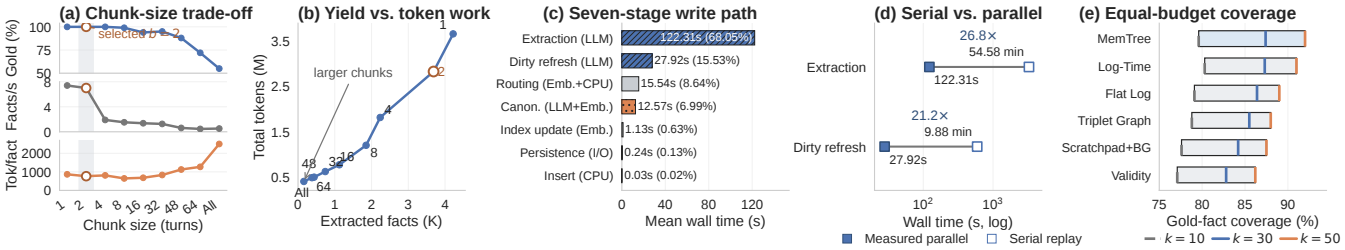

Figure 8: Extraction, write-path, and retrieval diagnostics. (a) Chunk-size coverage, throughput, and tokens/fact. (b) Extracted yield versus total token work. (c) Seven-stage write-path time. (d) Calibrated serial replay versus measured parallel execution. (e) Equal-budget coverage; each box spans $k = 10$ to $k = 50$, and the three ticks mark $k = 10, 30, 50$.

larger budgets, while flat logs avoid summary construction. Full curves and confidence intervals are in Appendix B.

Routing accuracy and scene stability. A manual audit finds 124/127 (97.6%) entity assignments valid but only 9/127 complete. Fragmentation lowers embedding-browse accuracy from 76.2% to 50.0% on LoCoMo and 78.1% to 67.6% on LongMemEval. Scene-threshold Jaccard mean/p05 is at least 0.9997/0.9996 (4B) and 0.9650/0.8829 (30B). These results support multi-scope placement and browse; full labels are in Appendix B.4.

We next study retrieval accuracy on a controlled 60-question LongMemEval subset, constructed by sampling 10 questions from each of the six question types. To reduce the influence of single-sample answer variance, we report pass@8 in the main text and provide the full question IDs in Appendix F. These experiments serve as retrieval diagnostics rather than replacements for the full-benchmark results in Section 5.2. We focus on two questions: whether multiple temporal tree views are necessary, and whether planner-guided tree browse improves evidence access once candidate trees have been recalled.

Structured views provide the measured gain; session trees preserve a fallback. Table 6 compares tree-family combinations under the same planner-guided LLM browse setting. Among single views, `session only` reaches **85.0%** pass@8, `scene only` reaches **81.7%**, and `entity only` reaches **63.3%**.

The `entity+scene` combination reaches **86.7%**, matching the full three-family setting on this subset. Thus entity and scene trees supply the measured structured-retrieval gain, while the session tree retains raw conversational evidence as a fallback; it adds no measured pass@8 gain on this subset.

Planner-guided LLM browse yields the highest measured accuracy. We next compare the six retrieval variants in Table 7 on the same 60-question subset. `flat-10` omits trees, `root-only` omits descent, `emb`/`llm` selects branches by embedding/LLM, and `+planner` adds per-tree subqueries. This comparison isolates how much the temporal tree structure and browse policy each contribute.

The first result is that neither flat retrieval nor root-only recall is sufficient. Flat top-10 retrieval reaches **78.3%**, while root-only recall is lower at **73.3%**. Together, these two baselines show that neither directly retrieving a flat top-10 set nor stopping at coarse root summaries can match the retrieval power of temporal MemTree browse.

Table 6: Tree-family ablation on the 60-question LongMemEval subset. We report 30B pass@8 under `llm+planner` browse.

Variant	30B pass@8
entity+scene+session	**86.7**
entity+scene	**86.7**
entity+session	85.0
session only	85.0
scene+session	83.3
scene only	81.7
entity only	63.3

The system must both preserve temporal scopes and descend within them to reach answer-bearing evidence.

The second result is that browse policy materially affects accuracy. Embedding-only browse already improves substantially over both flat and root-only baselines, reaching **85.0%**; pure LLM-guided browse reaches **88.3%**; and planner-guided LLM browse reaches the best result, **90.0%**. Once candidate trees have been recalled, planner-generated per-tree subqueries make browse more targeted by using each tree's root summary to specialize the search intent. This is especially useful in the LLM-guided setting, where the browser can exploit not only semantic similarity but also more structured intent such as temporal focus, state transitions, or finer-grained disambiguation. The added planner cost is acceptable here, because a single LLM call steers the subsequent per-tree LLM browse calls.

By contrast, planner guidance does not help embedding browse: planner-guided embedding browse slightly drops to **83.3%**. Under embedding-only descent, the rewritten query is still reduced to a vector similarity signal. Unlike LLM-guided branch selection, embedding similarity cannot reliably preserve richer browse intent such as time ranges, before/after relations, or transition constraints; it mostly captures semantic relatedness. In that setting, the extra planner call adds noticeable cost without a comparably targeted retrieval benefit. For this reason, the final system retains two operating modes: a lightweight embedding-only mode for latency-sensitive deployments, and a planner-guided LLM-guided mode for the high-accuracy setting.

Table 7: Retrieval and browse ablation on the 60-question LongMemEval subset. We report 30B pass@8 only.

Variant	30B pass@8
flat-10	78.3
root-only	73.3
emb	85.0
emb+planner	83.3
llm	88.3
llm+planner	**90.0**

7 RELATED WORK

Existing memory systems for LLM agents differ mainly in how memory is constructed, how persistent state is organized, and how that state is maintained as interactions accumulate. We review the work most related to MemForest along these three dimensions.

Memory Construction. A common direction is to build long-term memory by extracting salient information from interactions and storing it as persistent records. SeCom studies how memory units should be constructed and retrieved for conversational agents [25]. Mem0 emphasizes practical long-term memory for personalization and retrieval [3]. LightMem and MemoryOS introduce staged memory handling across short-, mid-, and long-term components, separating online use from offline consolidation and managed updates [9, 17]. Context-dependent memory frameworks highlight modular memory handling across interaction contexts [10]. MemForest focuses specifically on construction-path parallelism: it extracts short windows concurrently, consolidates them into canonical facts, and materializes indexes from that state.

Memory Organization. A second line of work studies how persistent memory should be organized after it is written. MemForest is aligned with this line, but differs in the representation it materializes: it organizes canonical facts into per-scope temporal trees so that retrieval can proceed from root summaries to leaf evidence within an explicit temporal hierarchy, rather than centering the design on recollection workflows or dynamically selected structures. EverMemOS models memory as a lifecycle of semantic consolidation and recollection [13]. A-Mem explores agentic memory organization, while H-Mem, HiMem, and Adapt investigate hybrid, hierarchical, or adaptive structures for long-context agents [15, 19, 36, 38, 40]. These systems move beyond flat memory stores and show that retrieval quality depends strongly on memory structure.

MemPalace and EverMemOS retain timestamped histories; LightMem and MemoryOS use recent/background stores; and Zep uses a bi-temporal graph [9, 13, 17, 22, 27]. These designs preserve history but shift work across construction, maintenance, and retrieval; Sections 6.2 and 5 compare controls and native systems.

Memory Maintenance. Another important question is how memory should evolve after it is written. MemForest differs from prior work by making a canonical, mergeable memory state with explicit temporal update semantics a first-class design goal: canonical facts are persistent state, summaries are derived artifacts regenerated incrementally, and this separation supports temporal preservation, incremental reorganization, and migration without replaying raw sessions. RMM treats memory as a reflective process, refining what is retained and how stored memory is later used [32]. At the other end of the design space, fidelity-first systems such as MemPalace preserve raw or near-verbatim history and defer abstraction to query time [22]. These approaches make different trade-offs between write-time processing and query-time reasoning, but neither centers on canonical, locally maintainable persistent state.

8 CONCLUSION

We presented MemForest, a long-context agent memory system that reframes persistent memory as a write-efficient temporal data-management problem. Canonical facts decouple persistent state from derived artifacts, while scoped, time-ordered MemTrees localize structural and summary maintenance and preserve current, historical, and transition evidence. Across three backbones, a MemForest variant leads LongMemEval-S; under Qwen it nearly matches EverMemOS on LoCoMo, while Gemma exposes model dependence. Controls position MemTree as a structured high-recall option among logs, intervals, scratchpads, and temporal graphs.

MemForest optimizes write latency rather than aggregate tokens: parallel extraction and level-wise refresh shorten the critical path but repeat prompt, schema, and structured-output overhead. Retained provenance supports targeted correction of extraction errors; automatic factuality verification remains future work.

REFERENCES

[1] Shubham Agarwal, Sai Sundaresan, Subrata Mitra, Debabrata Mahapatra, Archit Gupta, Rounak Sharma, Nirmal Joshua Kapu, Tong Yu, and Shiv Saini. 2025. Cache-Craft: Managing Chunk-Caches for Efficient Retrieval-Augmented Generation. *Proc. ACM Manag. Data* 3, 3, Article 136 (June 2025), 28 pages. doi:10.1145/3725273

[2] Bruno Becker, Stephan Gschwind, Thomas Ohler, Bernhard Seeger, and Peter Widmayer. 1996. An asymptotically optimal multiversion B-tree. *The VLDB Journal* 5, 4 (1996), 264–275.

[3] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413* (2025).

[4] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations (ICLR)*.

[5] DeepSeek. 2026. *Context Caching*. https://api-docs.deepseek.com/guides/kv_cache

[6] DeepSeek. 2026. *Models and Pricing*. DeepSeek. https://api-docs.deepseek.com/quick_start/pricing

[7] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).

[8] Ramez Elmasri, Gene TJ Wuu, and Yeong-Joon Kim. 1990. The time index: An access structure for temporal data. In *Proceedings of the 16th International Conference on Very Large Data Bases*. 1–12.

[9] Jizhan Fang, Xinle Deng, Haoming Xu, Ziyan Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. 2026. LightMem: Lightweight and Efficient Memory-Augmented Generation. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=dyJ0GWpjJB

[10] Pengyu Gao, Jinming Zhao, Xinyue Chen, and Long Yilin. 2025. An efficient context-dependent memory framework for llm-centric agents. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*. 1055–1069.

[11] Yubin Ge, Salvatore Romeo, Jason Cai, Raphael Shu, Yassine Benajiba, Monica Sunkara, and Yi Zhang. 2025. Tremu: Towards neuro-symbolic temporal reasoning for llm-agents with memory in multi-session dialogues. In *Findings of the Association for Computational Linguistics: ACL 2025*. 18974–18988.

[12] Tooraj Helmi. 2025. Decentralizing AI Memory: SHIMI, a Semantic Hierarchical Memory Index for Scalable Agent Reasoning. *arXiv preprint arXiv:2504.06135* (2025).

[13] Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, et al. 2026. EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning. *arXiv preprint arXiv:2601.02163* (2026).

[14] Guoyu Hu, Shaofeng Cai, Tien Tuan Anh Dinh, Zhongle Xie, Cong Yue, Gang Chen, and Beng Chin Ooi. 2025. HAKES: Scalable Vector Database for Embedding Search Service. *Proceedings of the VLDB Endowment* 18, 9 (2025), 3049–3062.

[15] Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. 2025. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 32779–32798.

[16] Paul Jackson and Jane Klobas. 2008. Transactive memory systems in organizations: Implications for knowledge directories. *Decision support systems* 44, 2 (2008), 409–424.

[17] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory os of ai agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 25972–25981.

[18] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. doi:10.1162/tacl_a_00638

[19] Mingfei Lu, Mengjia Wu, Feng Liu, Jiawei Xu, Weikai Li, Haoyang Wang, Zhengdong Hu, Ying Ding, Yizhou Sun, Jie Lu, et al. 2026. Choosing How to Remember: Adaptive Memory Structures for LLM Agents. *arXiv preprint arXiv:2602.14038* (2026).

[20] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 13851–13870.

[21] Mem0. 2026. Memory Benchmarks. https://github.com/mem0ai/memory-benchmarks. Judge prompts referenced by immutable repository commits.

[22] milla-jovovich. 2026. *MemPalace*. https://github.com/milla-jovovich/mempalace

[23] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. 1996. The log-structured merge-tree (LSM-tree). *Acta informatica* 33, 4 (1996), 351–385.

[24] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. MemGPT: towards LLMs as operating systems. (2023).

[25] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Xufang Luo, Hao Cheng, Dongsheng Li, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Jianfeng Gao. 2025. SeCom: On Memory Construction and Retrieval for Personalized Conversational Agents. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=xKDZAW0He3

[26] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*. 1–22.

[27] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956* (2025).

[28] Alireza Rezazadeh, Zichao Li, Ange Lou, Yuying Zhao, Wei Wei, and Yujia Bao. 2025. Collaborative memory: Multi-user memory sharing in llm agents with dynamic access control. *arXiv preprint arXiv:2505.18279* (2025).

[29] Alireza Rezazadeh, Zichao Li, Wei Wei, and Yujia Bao. 2025. From Isolated Conversations to Hierarchical Schemas: Dynamic Tree Memory Representation for LLMs. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=moXtEmCleY

[30] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D Manning. 2024. Raptor: Recursive abstractive processing for tree-organized retrieval. In *The Twelfth International Conference on Learning Representations*.

[31] Ji Sun, Guoliang Li, James Pan, Jiang Wang, Yongqing Xie, Ruicheng Liu, and Wen Nie. 2025. GaussDB-Vector: A Large-Scale Persistent Real-Time Vector Database for LLM Applications. *Proceedings of the VLDB Endowment* 18, 12 (2025), 4951–4963.

[32] Zhen Tan, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long Le, Yiwen Song, Yanfei Chen, Hamid Palangi, George Lee, et al. 2025. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 8416–8439.

[33] Gemma Team, Sherif El Abd, Vaibhav Aggarwal, Robin Algayres, Alek Andreev, Olivier Bachem, Ian Ballantyne, Cormac Brick, Victor Cărbune, Michelle Casbon, et al. 2026. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770* (2026).

[34] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=pZiyCaVuti

[35] Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chenchen Ling, et al. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. *arXiv preprint arXiv:2606.19348* (2026).

[36] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic Memory for LLM Agents. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=FiM0M8gcct

[37] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[38] Zihe Ye, Jingyuan Huang, Weixin Chen, and Yongfeng Zhang. 2026. H-Mem: Hybrid Multi-Dimensional Memory Management for Long-Context Conversational Agents. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7756–7775.

[39] Runjie Yu, Weizhou Huang, Shuhan Bai, Jian Zhou, and Fei Wu. 2025. AquaPipe: A Quality-Aware Pipeline for Knowledge Retrieval and Large Language Models. *Proc. ACM Manag. Data* 3, 1, Article 11 (Feb. 2025), 26 pages. doi:10.1145/3709661

[40] Ningning Zhang, Xingxing Yang, Zhizhong Tan, Weiping Deng, and Wenyong Wang. 2026. HiMem: Hierarchical Long-Term Memory for LLM Long-Horizon Agents. *arXiv preprint arXiv:2601.06377* (2026).

[41] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176* (2025).

[42] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2025. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems* 43, 6 (2025), 1–47.

[43] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 19724–19731.

[44] Wei Zhou, Xuanhe Zhou, Shaokun Han, Hongming Xu, Guoliang Li, Zhiyu Li, Feiyu Xiong, and Fan Wu. 2026. Are We Ready For An Agent-Native Memory System? *arXiv preprint arXiv:2606.24775* (2026).

A PROMPTS

A.1 LLM-as-Judge Prompts

LongMemEval Judge Prompt

Your task is to label an answer to a LongMemEval question as CORRECT or WRONG.
You will be given:
(1) question type
(2) question
(3) gold answer
(4) generated answer
Be generous with grading: (1) accept semantically equivalent answers; (2) accept equivalent relative and absolute time expressions; (3) accept more specific but consistent answers; (4) mark WRONG only for contradiction, missing key facts, answering a different question, or incorrect abstention.
Question type: {question_type}
Question: {question}
Gold answer: {gold_answer}
Generated answer: {generated_answer}
Return JSON only with {"label": "CORRECT"} or {"label": "WRONG"}.

LoCoMo Judge Prompt

Your task is to label an answer to a question as CORRECT or WRONG.
You will be given:
(1) question
(2) gold answer
(3) generated answer
Be generous with grading: (1) accept semantically equivalent answers; (2) accept equivalent relative and absolute time expressions; (3) accept more specific but consistent answers; (4) mark WRONG only for contradiction, missing key facts, non-answer, or incorrect abstention.
Question: {question}
Gold answer: {gold_answer}
Generated answer: {generated_answer}
Return JSON only with {"label": "CORRECT"} or {"label": "WRONG"}.

A.2 Retrieval and Answer Interfaces

Flat-item systems in the main tables expose a global final top-$k = 10$ context. We preserve each method's schema-aware answer interface because the returned objects differ: MemForest returns canonical facts and tree-derived evidence, EverMemOS returns episodic/recollection contexts, and other flat systems expose their native memory records. MemForest uses its native top-$k = 10$ tree-browse setting and fully expands selected tree units; the equal-flat-fact comparison is confined to the controlled retrieval study. Zep Local is not described as an equal ten-fact comparison: its native Graphiti search uses a limit of 10 per evidence class and serializes heterogeneous edges, nodes, episodes, and communities. Table A1 reports the resulting object counts and context-token lengths separately. Frozen answers are judged with `deepseek-v4-flash`, temperature zero, and thinking disabled.

The pinned EverMemOS rows preserve event timestamps in memory units and event logs and use its agentic retrieval mode. Round one performs hybrid retrieval and an LLM sufficiency check; if evidence is insufficient, LLM-refined queries drive a second retrieval round before final reranking. The released configuration keeps the final response budget at ten memory units.

Following Zhou et al. [44], the Zep Local row uses the released Graphiti/Neo4j adapter: Graphiti v0.24.1, Neo4j 5.26.2, raw-episode ingestion, and self-hosted generation and Qwen3-Embedding-0.6B endpoints. It is a reproducible local Graphiti realization, not Zep Cloud. The six-cell budget summary, its source manifest, and the aggregation script preserve the table's result-to-code path.

The native recipe therefore usually serializes approximately 20 heterogeneous objects, not ten flat facts. Small cross-backbone context-length differences arise because Graphiti extraction builds a different graph for each generative backbone.

We take both public prompts from the Mem0 benchmark repository [21]. LongMemEval uses root commit `7ba1bd3`. LoCoMo categories 1–4 use the tuned public prompt in commit `edcd6f1`. The headline scope follows the released public runners. At the pinned commit, the Mem0 harness uses the explicit setting

`CATEGORIES_TO_EVALUATE = [1,2,3,4].`

It marks the 446 category-5 questions as excluded from scoring and reports public headline results as 1,410/1,540 at top-200 and 1,273/1,540 at top-50.[1] The pinned EverMemOS runner likewise applies `filter_category:[5]`, and its reported LoCoMo evaluation contains 1,540 questions [13].[2] The public prompt assumes a non-empty reference answer, so it cannot define the policy for adversarial category 5; we retain the strict answerability judge for that category.

A.3 Strict/Public Judge Sensitivity

The tuned public LoCoMo template explicitly accepts dates within 14 days and durations within 50% in its date-tolerance rule.[3] The paired diagnostic freezes retrieval and generated answers. It changes only the judge prompt and contains 3 methods ×3 votes ×[172 LongMemEval questions ×2 judge arms +200 LoCoMo questions ×3 judge arms] = 8,496 successful judge calls, with zero judge errors. The reported temporal slices contain 122 LongMemEval and 150 LoCoMo questions within those broader frozen samples. On LoCoMo temporal, the corresponding public evaluation prompt increases EverMemOS, MemForest, and Mem0 by 17.33, 16.00, and 5.33 points. On LongMemEval temporal, the increases are 4.92, 2.46, and 4.92 points. The effect varies by benchmark and method, including MemForest. Under both prompts, the temporal ordering on both benchmarks remains MemForest, EverMemOS, then Mem0. The released stratified table reports every numerator and denominator.

A.4 LoCoMo Adversarial Category

Category 5 evaluates answerability instead of supplying the non-empty reference answer assumed by the released public LLM-judge runners. The headline comparison follows the released scope of 1,540 questions; we evaluate the 446 category-5 questions separately with the same strict answerability prompt for every method. MemoryOS obtains the highest category-5 score under all three backbones.

[1] https://github.com/mem0ai/memory-benchmarks/blob/edcd6f1d42400837b1fcb6997716f1769dc51a37/benchmarks/locomo/prompts.py; https://github.com/mem0ai/memory-benchmarks/blob/edcd6f1d42400837b1fcb6997716f1769dc51a37/README.md
[2] https://github.com/EverMind-AI/EverMemOS/blob/539db77d5cc804c875246e34611fd266bf8c1e5d/evaluation/config/datasets/locomo.yaml
[3] See `prompts.py`, line 228.

Table A1: Zep Local native retrieval budget. Object columns report the mean number serialized per question. Context tokens use the exact serialized context and one fixed Qwen tokenizer; they exclude the answer instruction and question.

Backbone	Benchmark	Edges	Nodes	Episodes	Communities	Context tokens avg.	Context tokens p95
Qwen3-4B	LongMemEval-S	5.00	5.00	9.99	0.00	3,269	4,542
Qwen3-4B	LoCoMo	5.00	4.97	10.00	0.00	1,360	1,626
Qwen3-30B	LongMemEval-S	5.00	5.00	9.99	0.00	3,125	4,412
Qwen3-30B	LoCoMo	5.00	5.00	10.00	0.00	1,210	1,413
Gemma-4-12B-IT	LongMemEval-S	5.00	5.00	9.99	0.00	3,105	4,364
Gemma-4-12B-IT	LoCoMo	5.00	5.00	10.00	0.00	1,168	1,348

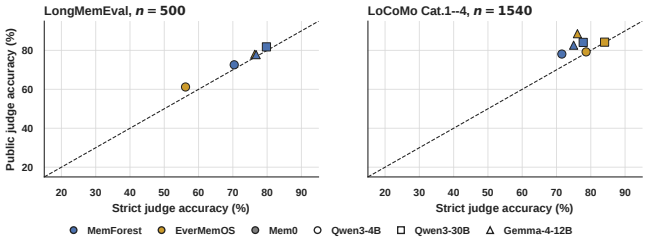

Figure A1: Paired strict/public-prompt sensitivity with frozen retrieval and answers. The diagonal denotes no change.

A.5 Mem0 Budget Occupancy and Answer-Prompt Coupling

This subsection is a separate sensitivity diagnostic for the public Mem0 Platform-v3 top-50/top-200 protocol. It is not the source of the 72.18% LongMemEval temporal value cited in R3-W6, which EverMemOS imports from the official MemOS baseline table.

The corrected Qwen3-30B Mem0 snapshot contains 121,594 memory units across 500 isolated LongMemEval-S stores, or 243.2 units per question on average. Top-10 exposes 4.18%, top-50 exposes 20.92%, and top-200 exposes 83.11% of a local store on average; top-200 exhausts 40/500 stores. On temporal questions it exposes 82.48% and exhausts 8/133 stores. We therefore treat the public top-200 result as a different-budget, broad high-coverage setting at this memory scale, not as a direct selective-retrieval comparison. The exact store-level occupancy distribution is released with the corrected Mem0 results.

Forcing one schema-neutral answer prompt changes schema interpretation, abstention, and evidence use because methods serialize heterogeneous objects differently. We therefore retain native schema-aware answer interfaces in the main protocol and use equal-budget retrieval coverage, rather than shared-prompt QA, as the representation-level diagnostic.

B TEMPORAL RETRIEVAL CONTROLS

B.1 Temporal Diagnostic Subset Map

Several temporal subsets serve distinct diagnostic purposes. Table B1 makes their relationship explicit; these diagnostic subsets do not replace the full-benchmark pass@1 results.

B.2 Fixed-Retrieval Dual-Time Control

To isolate context rendering from retrieval, both arms use the same top-30 fact IDs for all 321 official LoCoMo temporal questions. The answer backbone is Qwen3-30B and the paired answers are judged with the same public LoCoMo judge. Table B2 shows a net gain of ten correct answers; this targeted diagnostic does not replace the main-table protocol or establish that the complete LoCoMo gap is removed.

Table B2: Dual-time rendering with retrieval IDs held fixed.

Context arm	Correct / 321	Pass@1	Paired change
Fact text only	245	76.32%	–
Fact text + dual time	255	79.44%	+14 / −4

Exact paired McNemar $p = 0.0309$. Dual time preserves source conversation time, resolved event time, the original temporal expression, and its resolution base.

B.3 Log-TimeRerank Development Grid

Log-TimeRerank scores each fact using semantic similarity plus weighted lexical and temporal features. We freeze an 82-question development split and a disjoint 237-question held-out split by question ID. The predefined grid is lexical weight $\{0, 0.04, 0.08, 0.12\}$ and temporal weight $\{0, 0.5, 1.0, 1.5\}$. Table B3 reports development top-10 macro coverage. The selected pair $(0.12, 1.0)$ is the best point in this grid. Because 0.12 lies on the grid boundary, the search does not establish an optimum beyond the predefined range. Across the 16 settings, development coverage ranges from 76.67% to 82.60%.

Table B3: Log-TimeRerank development macro coverage (%).

Temporal \Lexical	0	0.04	0.08	0.12
0.0	77.89	77.48	79.92	81.79
0.5	77.89	79.11	80.33	82.20
1.0	76.67	77.89	79.11	**82.60**
1.5	76.67	77.48	78.70	80.57

Selection sensitivity. For transparency, the lightweight MemTree selector is also selected on the same development split, over 96 predefined combinations of tree-prior, novelty, and redundancy weights. Its top-10 development coverage ranges from 79.07% to 82.40%, and the selected setting is $(0.30, 0.00, 0.00)$. Both selectors are tuned on the same development split, selected once, and frozen

Table A2: Final evaluation protocol and sensitivity controls.

Component	Main setting	Sensitivity control	Reporting rule
Retrieval budget	Native top-10 units; MemForest expands tree units; Zep uses per-class limit 10	Equal flat-fact sweep; Mem0 occupancy at top-50/200	Report unit semantics and Zep object counts separately
Timestamp	Evaluated REST path writes benchmark time to `metadata.created_at/event_time`; SDK fallback uses `timestamp`; LoCoMo batches stay within one source session	Automated date-range and batch-coverage gates	Reject runtime dates and incomplete coverage
Answer prompt	Native schema-aware interface	Shared schema-neutral prompt	Treat shared-prompt scores as diagnostic
Judge	Benchmark-specific public prompts with `deepseek-v4-flash`	Strict paired judge	Main tables use the corresponding public prompt
LoCoMo Cat.5	Strict answerability judge	Not applicable	Report separately because the public prompt assumes non-empty gold

Table A3: LoCoMo category-5 strict answerability pass@1.

Method	Qwen3-4B	Qwen3-30B	Gemma-4-12B-IT
MemForest-Planner	29.60	35.87	10.09
MemForest-Embed	22.87	32.29	9.42
EverMemOS	23.77	19.73	8.97
LightMem	11.66	14.13	14.13
MemoryOS	**42.38**	**44.84**	**28.92**
MemPalace	14.57	15.92	16.59
Mem0	21.30	22.42	13.90
Zep Local	10.09	11.43	6.73

across all held-out budgets. The narrower observed MemTree range indicates lower sensitivity on these grids, while the held-out results establish that its advantage emerges when the pre-answer evidence budget is sufficient rather than at every k.

Table B4: Held-out macro gold-fact coverage across final evidence budgets (%).

Representation	5	10	20	30	50	100
Flat Log	71.8	79.1	84.7	86.4	89.0	94.6
Log-Time	71.8	80.3	84.7	87.3	91.0	95.1
Validity Interval	71.2	77.1	80.0	82.8	86.2	90.4
Scratchpad+Background	68.4	77.6	82.3	84.2	87.5	90.9
Triplet Graph	70.3	78.8	83.1	85.5	88.0	93.9
MemTree	71.0	79.6	84.0	87.4	92.0	96.4

B.4 Routing, Browse, and Fragmentation Diagnostics

Planner and Embed use different native candidate pools and the same lightweight semantic/time reranker; Planner adds one LLM browse call. The scene sweep rebuilds routes from frozen facts and embeddings, so its Jaccard score measures assignment stability.

Table B1: Temporal diagnostic subsets and their non-overlapping purposes.

# Questions	Purpose
321	Official LoCoMo raw-category-2 temporal set used for category accounting and the frozen-output error audit.
319	Category-2 questions with author-annotated gold-fact mappings for candidate coverage; two questions without reliable mappings are excluded.
82 + 237	Disjoint development and held-out partitions of the 319 mapped questions; only the 82-question split selects Log-TimeRerank and lightweight MemTree-selector weights.
249	Separate cross-benchmark LoCoMo/LongMemEval routing and fragmentation audit, rather than a further split of the 321-question set.
231	Supported mappings retained after independent review and author adjudication: 130 LoCoMo and 101 LongMemEval questions; 18 unsupported rows are excluded with reasons.

Table B5: Compact routing and retrieval diagnostics.

Diagnostic	Result	Boundary
Active entity routing	124/127 precision PASS; 3/127 exceptions; exact-all 9/127	Entity trees are a selective precision overlay; other scopes remain available
Planner vs. Embed	+0.49 to +2.37 pp macro coverage over $k = 5 \ldots 100$	Quality–cost comparison with one extra LLM browse call
Scene threshold	4B mean/p05 $\geq 0.9997/0.9996$; 30B $\geq 0.9650/0.8829$	Perturbs $\theta = 0.84, 0.92$ around default 0.88
Specific-tree fragmentation	4/130 LoCoMo; 37/101 LongMemEval	Evidence may remain reachable through union recall and fallback

The fragmentation audit retains 231/249 supported mappings after independent review and author adjudication. Under embedding browse, fragmented versus unfragmented questions score 2/4 (50.00%) versus 96/126 (76.19%) on LoCoMo and 25/37 (67.57%) versus 50/64 (78.13%) on LongMemEval. The released audit records separate model-assisted labels, independent review, author decisions, exclusions, and validation summaries. Together, the audit supports high-precision active routes but not a single-route design: incomplete entity coverage and the LongMemEval fragmentation gap motivate redundant scope assignment, union recall, and multi-tree browse.

B.5 Extraction and Tree-Shape Sensitivity

The following configuration diagnostics support the defaults used by MemForest without using benchmark pass@1 to select the reported operating points.

C CHUNK-SIZE DIAGNOSTIC FOR RAW-FACT EXTRACTION

We study extraction chunk size in a separate diagnostic setting, since this operating-point choice is not the main contribution of MemForest and is therefore deferred from the main ablation section. The purpose of this experiment is to examine how raw-fact extraction degrades as chunk granularity increases.

To create a controlled stress setting, we manually assemble long sessions by concatenating multiple original conversations, then run the same raw-fact extraction pipeline under different chunk presets. This setup is therefore a diagnostic stress test rather than the benchmark's native dialogue layout, and is intended to expose how the extraction pipeline behaves as chunk granularity is varied.

We evaluate each setting using **Gold Coverage**, a diagnostic retention metric rather than an end-to-end QA metric. For each question, we identify its gold supporting turn range and the key answer-bearing span within that range, such as an entity, date, number, or short attribute phrase. A question is covered if at least one extracted fact produced from a chunk intersecting the gold range still preserves that key span after normalization.

Table C1 shows a clear constrained operating point. Whole-session extraction is both the least faithful and one of the least efficient settings, while chunk sizes beyond 8 turns show visible fidelity degradation. Very small chunks preserve answer-bearing information, but 2-turn extraction provides the best overall balance: it preserves 100% Gold Coverage, remains near the best throughput regime, and improves token efficiency over 1-turn extraction. We therefore use 2-turn extraction as the default write-path setting in the main system.

MemTree branching factor. Figure C1 separates the two constraints behind the default branching factor. Increasing k makes the tree shallower, but eventually asks one autoregressive summary call to preserve too many child facts. The selected moderate operating region balances dependency depth against summary retention; it is not tuned on the benchmark answer judge.

Table C1: Chunk-sweep study for raw-fact extraction on the assembled-long-session benchmark.

Preset	Gold Coverage (%)	Facts/s	Token/fact
1-turn	100	7.40	868
2-turn	100	7.00	767
4-turn	100	1.90	811
8-turn	99	1.52	647
16-turn	94	1.37	682
32-turn	95	1.26	832
48-turn	88	0.62	1127
64-turn	72	0.46	1271
whole	55	0.52	2505

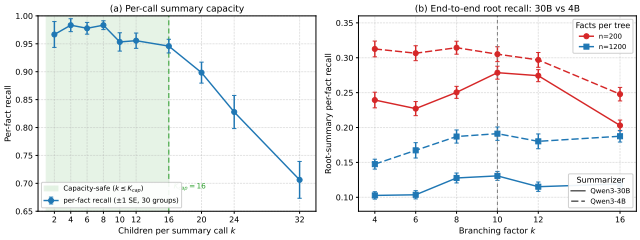

Figure C1: MemTree branching-factor diagnostics. (a) Per-fact retention in one summary call remains high through a moderate number of children and then declines. (b) End-to-end root-summary recall peaks at moderate branching factors across two tree sizes and two summarizers. Here k denotes tree branching factor, not final retrieval top-k; these controlled summary-retention measurements are not benchmark pass@1.

D SYSTEM COST, SCALING, FRESHNESS, AND MIGRATION

D.1 Matched Request-Level Token Probe

The same three-stream trace averages 435.7K input and 451.9K output tokens over 258.3 extraction calls, and 177.2K input and 77.6K output tokens over 320.0 dirty-refresh calls. In matched historical service logs, 46 stable windows with one running request, no queued request, and no prompt processing sustain a median 140.8 output tokens/s (p10–p90: 140.2–145.9). Request-level fits give effective prompt rates of 2.67K–21.85K tokens/s, depending on prefix reuse. Applying $T_{\text{serial}} = T_{\text{in}}/r_{\text{prefill}} + T_{\text{out}}/r_{\text{decode}} + n_{\text{req}} t_{\text{fixed}}$ defines a throughput-calibrated serial replay of the logged request stream. Across the observed prefill-rate range, replay takes 52.76–56.41 minutes for extraction and 9.23–10.52 minutes for dirty refresh. Figure 8(c–d) uses the respective midpoints, 54.58 and 9.88 minutes, versus measured parallel walls of 122.31 and 27.92 seconds (26.8× and 21.2× speedups). This replay holds the request stream fixed and sets the maximum in-flight request count to one; eager per-insert refresh would instead change the request sequence.

Under this self-hosted vLLM scheduler, prefix reuse reduces MemForest prefill relative to EverMemOS but not Mem0. Request length alone does not explain the result: MemoryOS has the largest p95 lengths, while Zep Local makes far more requests. MemForest's

Table D1: Self-hosted vLLM APC request diagnostic over 20 matched LoCoMo messages. Token p95 uses the nearest-rank definition. Cached input is measured from `prompt_tokens_details.cached_tokens`: it is vLLM avoided prefill, not a provider-billed cache class or a semantic prompt partition.

Method	Req.	Input avg.	Input p95	Uncached avg.	Uncached p95	Output avg.	Output p95	Cache share
MemForest	19	1,196	2,482	498	2,482	267	670	58.38%
EverMemOS	29	1,098	1,594	897	1,594	187	380	18.29%
Mem0	13	1,306	1,741	515	1,520	155	459	60.59%
MemoryOS	40	482	2,837	366	1,941	167	1,474	24.07%
Zep Local	190	723	1,536	317	1,124	58	195	56.14%

Table D2: MemForest write-stage breakdown (179.74 seconds total).

Stage	Time	Share	Mean calls	Main work
Extraction	122.31 s	68.05%	258.3 LLM	LLM
Canonicalize/dedup	12.57 s	6.99%	31.3 LLM + 49 emb.	Hybrid
Routing	15.54 s	8.64%	5.7 emb.	Embedding + CPU
Bucketize/insert	0.03 s	0.02%	0	CPU
Dirty refresh	27.92 s	15.53%	320.0 LLM	LLM
Index update	1.13 s	0.63%	3.0 emb.	Embedding
Persistence	0.24 s	0.13%	0	I/O

Table D3: Mean provider-billed token use and direct cost over three probes.

Method	Cached in	Miss in	Output	Total	Cost (¢)	Eff.
MemForest	15.45k	7.51k	3.64k	26.60k	0.212	5.28×
EverMemOS	3.37k	24.57k	3.45k	31.40k	0.442	2.53×
Mem0	13.65k	4.44k	1.27k	19.37k	0.102	11.00×
MemoryOS	1.88k	8.84k	2.44k	13.15k	0.193	5.81×
Zep Local	47.66k	66.77k	6.06k	120.48k	1.118	1.00×

Cost efficiency is normalized to Zep Local. Costs use provider-returned billable token classes and the official non-thinking DeepSeek-V4-Flash price [6].

Table D4: Cold-start versus verified warm-cache DeepSeek billing. Hit share is provider-reported hit input divided by total input; cost is US cents per 20-message native probe. Warmups are excluded from both columns.

Method	Cold hit	Warm hit	Cold cost	Warm cost
MemForest	0.00%	67.28%	0.413	0.212
EverMemOS	1.83%	12.06%	0.479	0.442
Mem0	50.60%	75.47%	0.166	0.102
MemoryOS	7.27%	17.52%	0.225	0.193
Zep Local	38.60%	41.65%	1.138	1.118

Table D5: Embedding retrieval as frozen state grows.

Forests	Facts	Trees	Root rows	p50	p95
1	1,428	250	210	37.63 ms	41.79 ms
2	2,694	463	369	39.14 ms	43.58 ms
4	5,553	950	754	39.10 ms	44.13 ms
8	10,781	1,868	1,455	39.24 ms	42.23 ms

measured advantage comes from parallel extraction cells and same-level, cross-tree refresh waves. Caching reduces repeated prefill but not dynamic prefill or autoregressive decoding; the self-hosted deployment has no per-token API bill.

The DeepSeek billing probe measures a separate provider-billed cache metric. We verified that the ten MemForest extraction requests in the Qwen/vLLM and DeepSeek probes have the same ten prompt hashes and the same fixed 6,014-character system prefix. They are submitted as one burst (9.7 ms and 17.3 ms start-time spans, respectively). vLLM reports cache hits for nine of these ten requests (12,944 avoided-prefill tokens), whereas DeepSeek reports zero. This is consistent with backend semantics rather than a lost accounting field: DeepSeek requires a complete persisted prefix unit, its common-prefix example first identifies the prefix after two requests complete, and cache construction can take seconds [5]. Therefore Table D1 characterizes local serving work, while Table D3 alone determines the reported API bill; cache shares and rankings are not compared across the two backends.

The cold-to-warm change is largest for MemForest because its long extraction instruction is shared by one concurrent first wave: vLLM can reuse scheduled blocks within that wave, while DeepSeek cannot bill a hit until the prefix unit has persisted. We use the verified warm-cache result for steady-state cost efficiency and report cold-start behavior separately.

For API billing, we probe three distinct LoCoMo conversations (`conv-26`, `conv-42`, and `conv-43`), truncate each to 20 messages, and rebuild every method from an empty store. Two additional conversations, `conv-47` and `conv-48`, warm each method under the same isolated provider `user_id` and are excluded from measured totals. We verify cache-eligible fixed templates; the second native warmup produces a nonzero hit for every method. All 15 measured cells complete without failed requests. Table D3 reports an unweighted mean because every probe has the same message count; per-conversation records are released with the probe. MemoryOS makes 34–38 requests and consumes 9.09k–18.93k total tokens because its LLM-generated session grouping changes how often the heat-triggered profile/knowledge update fires.

D.2 Frozen-Index Query Scaling

The sweep uses fixed top-10 embedding browse and excludes planner calls, answer generation, index loading, and memory construction. A separate synthetic structural sweep varies both facts per tree

Table D6: Session-level entity/scene-tree write concentration.

Scope	Trees/sess.	Facts/tree mean/med/p95/max	Within	Across
All trees	7.60	5.39/3/19/53	67.1%	89.3%
Specific trees	6.65	5.04/2/21/53	63.7%	16.6%

"Specific" excludes the global `entity:user` fallback. Within is the share of session–tree buckets containing at least two facts; Across is the share of within-user session pairs sharing at least one tree.

and number of trees. Across the two complete 500-question Qwen builds, the audit covers 250,118 trees; observed maximum height is four and no tree violates the conservative bounded-fan-out height check.

D.3 Freshness Contract and Write Concentration

Section 4.5 states the online tree-level concurrency contract. Each asynchronous B+ tree has an independent reader–writer lock: insertion and refresh are writers, whereas range query, browse primitives, and state export are readers. Distinct trees and concurrent readers therefore proceed independently; only a writer and another operation on the same tree conflict. The online auto-update path refreshes dirty trees and root vectors before returning. Snapshot save exports each tree under its read lock, writes a temporary directory, and renames the completed snapshot into place. The released source snapshot contains the evaluated lock and scheduler implementation.

Session-level write concentration. We attribute canonical facts to their first source session and join them to the final entity and scene trees in the same three representative LongMemEval streams used by Figure 8(c–d). Table D6 reports all 147 sessions. A within-session collision is a session–tree bucket containing more than one fact. Across-session overlap compares session pairs only within the same independent user stream. The released audit directory contains the 1,117 session–tree rows, manifest, summary, and regeneration script.

Repeated same-session assignments do not trigger one LLM refresh per fact: each tree is marked dirty once and shared ancestors are summarized once per flush. Across sessions, 16.6% of pairs share a specific entity/scene tree; including the global `entity:user` fallback raises overlap to 89.3%, making it the primary lock hotspot. This fallback can be partitioned into independently built subforests and consolidated through the same migration merge evaluated in Section 5.6.

Provenance links extraction errors to affected leaves and derived artifacts for targeted deletion and refresh.

D.4 Migration-State Scaling

Sequential replay and migration merge produce comparable persistent states: facts differ by less than 1% and trees by at most about 8%. The procedures need not be bit-identical because extraction, routing, and summary refresh contain model or heuristic decisions, but the measured migration speedup does not come from collapsing or discarding memory state.

Table D7: Memory scale under sequential write and migration merge.

N	Seq Facts	Mig Facts	Seq Trees	Mig Trees
1	1,435	1,435	249	249
2	2,692	2,716	375	405
3	4,061	4,098	562	607
4	5,540	5,590	733	792
5	6,860	6,922	895	967
6	8,101	8,174	1,044	1,128
7	9,426	9,511	1,200	1,296
8	10,705	10,801	1,362	1,472

E DETAILED WRITE-PATH AND PARALLELISM ANALYSIS

This appendix details the released write-path dependencies summarized in Section 5.4. One complete memory instance contains N method-native input units and has a per-instance LLM worker budget P. Actual worker limits are run-time configuration parameters rather than complexity constants. A star denotes dependency-feasible parallel calls whose released per-instance ingestion path is serial. Because APIs batch dialogue differently, N denotes native units rather than equal-size text chunks; wall-clock latency is measured on complete question or conversation stores.

The comparison reports autoregressive-LLM dependency depth, not constant-time token work. Prompt/output length, CPU and database work, serving saturation, and persistence remain measurable costs. Extraction creates the method's first persistent memory representation; maintenance reconciles, summarizes, or indexes it. A stage is LLM-based when an autoregressive result lies on its commit path.

E.1 Method-Specific Dependency Boundaries

Mem0. One add extracts facts, retrieves candidates, and uses a second LLM to choose add/update/delete/no-op actions. Isolated extraction prompts are independent, but the evaluated adapter awaits the complete stateful add before issuing the next. Concurrent adds can reconcile against and mutate the same old record; decoupling extraction would require ordered commits, per-record locking, or optimistic validation. We therefore report the implemented $O(N)$ add chain while marking extraction as dependency-feasible parallel work.

MemoryOS.. Raw QA append is non-LLM, while queue eviction can trigger page continuity, meta-summary, profile, and knowledge updates. Profile and knowledge analysis already use two workers within a trigger, but both depend on selected mid-term pages and same-user queue/profile commits remain ordered. A hot profile can also make a triggered LLM input grow with history.

EverMemOS.. Streaming boundary detection observes unresolved conversation history, so later boundaries depend on earlier decisions and one conversation has $O(N)$ ordered extraction depth. Once MemCells are frozen, embedding and indexing distinct cells

Table E1: Detailed write-path decomposition. Complexity denotes logical dependency depth. A star marks dependency-feasible parallel calls whose released per-instance ingestion path is serial.

System	Extraction path	LLM	Maintenance path	LLM
Mem0	History-independent extraction: $O(\lceil N/P \rceil)^*$; released add loop $O(N)$	Yes	Candidate search and reconciliation per add; $O(N)$ ordered commits	Yes
MemoryOS	Raw QA append; $O(N)$ ordered queue operations	No	At most N triggered page/meta-summary and profile updates, each with up to $O(N)$ profile input	Yes
EverMemOS	Boundary detection and MemCell formation; $O(N)$ ordered stream	Yes	Embedding/index persistence for at most N emitted cells; $O(N)$ work	No
LightMem	Post-segmentation extraction: $O(\lceil N/P \rceil)^*$; released add loop $O(N)$	Yes	Frozen-snapshot consolidation: $O(\lceil N/P \rceil)$ independent LLM waves	Yes
MemPalace	Deterministic chunk/drawer formation; $O(N)$ non-LLM work	No	Embedding and append/index insertion; $O(N)$ work, no semantic rewrite	No
Zep Local	Node/edge extraction per episode; $O(N)$ ordered episode chain	Yes	$O(N)$ ordered LLM chain; backend search $\sum_i g(G_i)$, index- and graph-dependent	Yes
MemForest	Implemented gather plus semaphore; $O(\lceil N/P \rceil)$ request waves	Yes	$\sum_{\ell=0}^{H} \lceil D_\ell/P \rceil = O(\lceil D/P \rceil + H)$ level-ordered waves	Yes

add no history-dependent LLM call and can be scheduled concurrently. Optional foresight/profile extractors are disabled in the evaluated configuration.

LightMem. Buffer accumulation and segmentation are ordered, but emitted segments could be extracted in bounded parallel waves; the released benchmark runner instead awaits each `add_memory`. Its maintenance workers summarize independent entries from one frozen candidate snapshot and serialize only non-LLM mutations under a write lock. Thus the evaluated extraction path is serial, whereas its snapshot-based LLM maintenance already uses $P = 5$ workers.

MemPalace. Chunk and drawer creation, embedding, and index insertion use no autoregressive LLM and perform $O(N)$ work. Preparation and embedding can be parallelized subject to ordered identifier/timestamp commit. Zero LLM depth does not imply zero CPU or database cost, and the method does not perform LLM temporal reconciliation on write.

Zep Local / Graphiti. Native `add_episode` extracts and resolves nodes/edges, invalidates superseded facts, and commits graph objects. Episodes in one namespace are awaited sequentially because later resolution observes the updated graph; within-episode operations and independent namespaces can still run concurrently. We denote indexed candidate-search and graph-query work before episode i by $g(G_i)$ because it depends on graph size, indexes, and query plans. The ordered LLM chain buys versioned facts, provenance, and topology and is not an asymptotic claim about every Graphiti deployment.

MemForest. Extraction cells do not read accumulated memory and take $O(\lceil N/P \rceil)$ request waves. For M_s new facts in scope s, structural CPU work is $\sum_s O(M_s \log N_s)$. Dirty-summary regeneration performs $O(D)$ total LLM work but allows same-level and cross-tree nodes to run together. If D_ℓ is the number of dirty summaries at level ℓ and H is the maximum touched-tree height, the exact request-wave depth is

$$\left\lceil \frac{N}{P} \right\rceil + \sum_{\ell=0}^{H} \left\lceil \frac{D_\ell}{P} \right\rceil = O\left(\left\lceil \frac{N+D}{P} \right\rceil + H \right).$$

For a complete build with bounded extraction yield and balanced bounded fan-out, $D = O(N)$ and $H = O(\log N)$, so this becomes $O(\lceil N/P \rceil + \log N)$. Routing, structural insertion, index update, and persistence are non-LLM. With fixed P, the LLM request-wave bound remains $O(N)$, while non-LLM structural work remains localized to inserted facts and touched trees. The contribution is bounded parallel waves and localized dependency, not sublinear total work or end-to-end latency.

The table analyzes schedules and state dependencies in the released paths evaluated here. Redesigned parallel schedulers are outside its scope.

F LONGMEMEVAL DIAGNOSTIC SUBSET FOR ABLATION

For the retrieval ablations in Section 6.2, we construct a balanced 60-question diagnostic subset by sampling 10 questions from each LongMemEval question type. Table F1 lists the full question IDs.

Table F1: Question IDs in the 60-question LongMemEval diagnostic subset (10 per type).

Question Type	Question IDs
Knowledge Update	01493427, 031748ae, 0977f2af, 18bc8abd, 1cea1afa, 4d6b87c8, 69fee5aa, 852ce960, a1eacc2a, f9e8c073
Multi-Session	1a8a66a6, 60bf93ed_abs, 73d42213, 81507db6, 88432d0a_abs, aae3761f, d682f1a2, e5ba910e_abs, gpt4_d84a3211, gpt4_f2262a51
Single-Session (Assistant)	0e5e2d1a, 1568498a, 16c90bf4, 561fabcd, 6222b6eb, 7161e7e2, 71a3fd6b, 7a8d0b71, 8cf51dda, c7cf7dfd
Single-Session (Preference)	06f04340, 0edc2aef, 195a1a1b, 1a1907b4, 1d4e3b97, 35a27287, 6b7dfb22, 75832dbd, a89d7624, caf03d32
Single-Session (User)	001be529, 311778f1, 545bd2b5, 5d3d2817, 60d45044, 94f70d80, af8d2e46, c5e8278d, caf9ead2, faba32e5
Temporal Reasoning	4dfccbf7, 982b5123, e4e14d04, gpt4_18c2b244, gpt4_1e4a8aec, gpt4_2487a7cb, gpt4_68e94287, gpt4_7a0daae1, gpt4_e072b769, gpt4_f420262d